Contents

1 The Java Platform **1**
- 1.1 What Java Is: Write Once Run Anywhere . . . 1
- 1.2 The JVM, JDK, JRE . . . 1
- 1.3 CPython vs JVM . . . 2
- 1.4 Bytecode and Class Files . . . 2
- 1.5 JIT Compilation . . . 3
- 1.6 GraalVM and Native Image . . . 3
- 1.7 Java Version History . . . 4
- 1.8 Where Java Is Used Today . . . 4

2 Environment and Tooling **5**
- 2.1 Installing OpenJDK . . . 5
- 2.2 The javac Compiler and java Runtime . . . 5
- 2.3 IntelliJ IDEA vs Eclipse vs VS Code . . . 6
- 2.4 Maven . . . 6
- 2.5 Gradle . . . 7
- 2.6 Creating a Runnable JAR . . . 7

3 Language Foundations **8**
- 3.1 Program Structure . . . 8
- 3.2 Primitive Types in Full . . . 8
- 3.3 Primitives vs Reference Types . . . 9
- 3.4 Literals . . . 9
- 3.5 Wrapper Classes . . . 9
- 3.6 var Keyword (Java 10+) . . . 10
- 3.7 final Keyword . . . 10

4 Operators and Control Flow **11**
- 4.1 Operators and Precedence . . . 11
- 4.2 Integer Overflow . . . 11
- 4.3 Short-Circuit Evaluation . . . 12
- 4.4 Switch Statement vs Switch Expression . . . 12
- 4.5 Enhanced For Loop . . . 12
- 4.6 Labeled break and continue . . . 13
- 4.7 Ternary Operator . . . 13

5 Methods **14**
- 5.1 Method Signatures . . . 14
- 5.2 Pass-by-Value . . . 14
- 5.3 Method Overloading . . . 14
- 5.4 Varargs . . . 15
- 5.5 Static Methods vs Instance Methods . . . 15
- 5.6 Recursion . . . 15

6 Arrays **17**
6.1 Array Declaration, Initialization, Memory Layout . . . 17
6.2 Multi-dimensional Arrays . . . 18
6.3 System.arraycopy vs Arrays.copyOf vs clone . . . 18
6.4 Arrays Utility . . . 19
6.5 Array vs ArrayList . . . 20

7 Strings in Depth **22**
7.1 String Immutability . . . 22
7.2 String Comparison . . . 23
7.3 Key Methods . . . 23
7.4 StringBuilder . . . 24
7.5 StringBuffer vs StringBuilder . . . 25
7.6 Text Blocks (Java 15+) . . . 26
7.7 String.format and printf . . . 26
7.8 Regular Expressions with Pattern and Matcher . . . 27

8 Object-Oriented Programming **28**
8.1 Classes and Objects . . . 28
8.2 Constructors . . . 28
8.3 Access Modifiers . . . 29
8.4 Encapsulation . . . 29
8.5 static Fields and Methods . . . 29
8.6 Inheritance . . . 30
8.7 Method Overriding . . . 30
8.8 super Keyword . . . 31
8.9 final Classes and Methods . . . 31
8.10 Abstract Classes . . . 31
8.11 Interfaces . . . 31
8.12 Multiple Interface Implementation vs Single Class Inheritance . . . 32
8.13 Polymorphism . . . 32
8.14 Casting . . . 32
8.15 instanceof with Pattern Matching (Java 16+) . . . 33
8.16 The Object Class . . . 33
8.17 Records (Java 16+) . . . 33
8.18 Sealed Classes (Java 17+) . . . 34
8.19 Enums . . . 34

9 Memory and Object Lifecycle **35**
9.1 The Heap and the Stack . . . 35
9.2 Object Creation Cost . . . 35
9.3 Garbage Collection . . . 35
9.4 Generational GC . . . 36
9.5 GC Algorithms . . . 36
9.6 Soft, Weak, and Phantom References . . . 37
9.7 Memory Leaks in Java . . . 37
9.8 try-with-resources and AutoCloseable . . . 38

10 Generics **39**
10.1 Why Generics Exist . . . 39
10.2 Generic Classes and Methods . . . 39
10.3 Bounded Type Parameters . . . 39
10.4 Wildcards . . . 40
10.5 PECS Principle . . . 40
10.6 Type Erasure . . . 40
10.7 Raw Types . . . 41
10.8 Generic Arrays . . . 41

11 Collections Framework **42**
11.1 The Collection Hierarchy . . . 42
11.2 ArrayList . . . 42
11.3 LinkedList . . . 43
11.4 ArrayDeque . . . 43
11.5 HashSet and HashMap . . . 44
11.6 LinkedHashSet and LinkedHashMap . . . 45
11.7 TreeSet and TreeMap . . . 45
11.8 PriorityQueue . . . 45
11.9 Comparable vs Comparator . . . 46
11.10 Collections Utility Class . . . 46
11.11 List.of, Set.of, Map.of (Java 9+) . . . 47
11.12 Arrays.asList Trap . . . 47

12 Lambdas and the Stream API **48**
12.1 Anonymous Classes Before Lambdas . . . 48
12.2 Functional Interfaces . . . 48
12.3 Built-in Functional Interfaces . . . 48
12.4 Lambda Syntax . . . 49
12.5 Method References . . . 49
12.6 Closure Semantics . . . 49
12.7 Stream Creation . . . 50
12.8 Intermediate Operations . . . 50
12.9 Terminal Operations . . . 51
12.10 Collectors in Depth . . . 51
12.11 Optional<T> . . . 52
12.12 Parallel Streams . . . 52

13 Exception Handling **53**
13.1 The Throwable Hierarchy . . . 53
13.2 Checked vs Unchecked . . . 53
13.3 try/catch/finally . . . 53
13.4 Multi-catch . . . 54
13.5 try-with-resources . . . 54
13.6 throw and throws . . . 54
13.7 Exception Chaining . . . 54
13.8 Custom Exceptions . . . 54
13.9 Common Anti-patterns . . . 55

14 Input and Output **56**
14.1 java.io Overview . . . 56
14.2 FileInputStream, FileOutputStream, etc. . . . 56
14.3 PrintWriter and System.out . . . 56
14.4 java.nio.file . . . 56
14.5 Serialization . . . 57
14.6 JSON with Jackson . . . 57
14.7 JSON with Gson . . . 57

15 Concurrency **58**
15.1 Thread Creation . . . 58
15.2 Thread Lifecycle . . . 58
15.3 Race Conditions . . . 58
15.4 synchronized Keyword . . . 58
15.5 volatile Keyword . . . 59
15.6 Deadlock . . . 59
15.7 java.util.concurrent ExecutorService . . . 59
15.8 Future and Callable . . . 60
15.9 ThreadPoolExecutor Internals . . . 60
15.10 Atomic Classes . . . 60
15.11 ConcurrentHashMap . . . 61
15.12 CountDownLatch, CyclicBarrier, Semaphore, Phaser . . . 61
15.13 BlockingQueue . . . 61
15.14 CompletableFuture . . . 61
15.15 Virtual Threads (Java 21) . . . 62

16 The Module System (Java 9+) **63**
16.1 Why Modules . . . 63
16.2 module-info.java . . . 63
16.3 Named, Automatic, Unnamed Modules . . . 63
16.4 Classpath vs Module Path . . . 64
16.5 –add-opens and –add-exports . . . 64
16.6 Strong Encapsulation . . . 64

17 Modern Java Features **65**
17.1 Pattern Matching for instanceof . . . 65
17.2 Switch Expressions with Patterns . . . 65
17.3 Sealed Classes and Exhaustive Switches . . . 65
17.4 Records Deep Dive . . . 65
17.5 Text Blocks . . . 66
17.6 String Templates (Java 21 Preview) . . . 66
17.7 Sequenced Collections (Java 21) . . . 66

18 Standard Library **67**
18.1 java.lang . . . 67
18.2 java.time Date/Time API . . . 67
18.3 java.math BigDecimal . . . 67
18.4 java.net.http HttpClient (Java 11+) . . . 67
18.5 java.util.regex . . . 67

19 Testing and Build **68**
19.1 JUnit 5 . . . 68
19.2 JUnit 5 Assertions . . . 68
19.3 Parameterized Tests . . . 68
19.4 Mockito . . . 68
19.5 Test-Driven Development . . . 68
19.6 Maven Test Lifecycle . . . 68
19.7 Code Coverage with JaCoCo . . . 69

20 Performance and Best Practices **70**
20.1 JVM Startup and Warm-up . . . 70
20.2 JVM Flags . . . 70
20.3 JMH (Java Microbenchmark Harness) . . . 70
20.4 Profiling . . . 70
20.5 String Concatenation Performance . . . 70
20.6 Boxing Overhead . . . 71
20.7 Effective Java Principles . . . 71
20.8 Common Design Patterns . . . 71
20.9 Naming Conventions . . . 71

Chapter 1

The Java Platform

1.1 What Java Is: Write Once Run Anywhere

Java was designed from the ground up to solve the platform-dependency nightmare that plagued C and C++ applications. In traditional compiled languages, you build a separate binary for each operating system and processor architecture. Java replaces that model with a two-step process: compile source code to architecture-neutral bytecode, then run that bytecode on a Java Virtual Machine (JVM) that has been tailored for the target platform. The promise is encapsulated in the slogan "Write Once, Run Anywhere" (WORA).

The bytecode is not interpreted from source. It is a compact, pre-verified intermediate representation stored in `.class` files. The JVM loads these class files, verifies their structural integrity (no stack underflows, valid branch targets, etc.), and then either interprets the bytecode or compiles it to native machine code on the fly. Because the JVM presents a uniform execution model (a stack-based abstract machine with a fixed instruction set), the same `.class` file will behave identically on Linux x86, Windows ARM, and macOS.

The reality is more nuanced. Thread scheduling policies differ between operating systems. The accuracy of `System.nanoTime()` varies. File locking semantics diverge on Windows and Unix. Graphics subsystems (AWT, Swing, JavaFX) rely on native peers that behave slightly differently. Still, for the server-side, headless workloads that dominate modern Java usage, WORA holds almost perfectly. A Spring Boot fat JAR assembled on a macOS developer laptop runs unchanged in a Linux container in production. The only requirement is that the JVM major version is compatible.

Portability comes at a cost: startup time. The JVM performs class loading, verification, static initialisation, and interpretation before reaching peak throughput. This is why Java traditionally felt "slow" to start. The trade-off is that after warm-up, the JIT-compiled code can rival or surpass ahead-of-time compiled languages because it exploits runtime profiling that static compilers cannot see. GraalVM Native Image removes the startup cost by compiling the application ahead of time into a standalone executable, but it gives up some of the JVM's dynamic optimisation capabilities.

1.2 The JVM, JDK, JRE

The three terms are nested layers of a Russian doll, but they are often confused.

- **JVM** – The Java Virtual Machine. This is the executable that runs bytecode. It contains the bytecode interpreter, the just-in-time compilers (C1 and C2 in HotSpot), the garbage collector, and the threading subsystem. The JVM itself does not include the standard library classes; it only knows how to load and execute classes.
- **JRE** – Java Runtime Environment. The JRE is the JVM bundled with the standard class libraries (`java.lang`, `java.util`, `java.io`, etc.) and configuration files. It provides everything needed to *run* a Java application. Historically you could download a standalone JRE, but since Java 11, Oracle and OpenJDK distributions no longer provide a separate JRE. The assumption is that you can create a custom runtime image using `jlink` from the JDK.

- **JDK** – Java Development Kit. The JDK contains the JRE plus the development tools: the compiler (`javac`), the archiver (`jar`), the documentation generator (`javadoc`), the debugger (`jdb`), and diagnostic tools (`jcmd`, `jstack`, `jmap`). When you install any modern Java distribution, you are installing a JDK.

In a production container, you would typically not ship the full JDK to reduce image size. Instead, you use `jlink` to create a minimal runtime image containing only the modules your application requires. This custom image includes a JVM and the needed standard library modules, effectively a tailored JRE.

1.3 CPython vs JVM

CPython, the reference Python implementation, is an interpreter written in C. It reads Python source code (or pre-compiled `.pyc` bytecode) and executes it on a virtual machine that is much closer to the high-level semantics of Python. The JVM, by contrast, was designed for static typing. Java bytecode contains explicit type information; the JVM can rely on the fact that a method will always receive a `String` in a particular slot. CPython's bytecode is untyped and must constantly dispatch based on the dynamic type of the operands.

The differences have profound performance implications:

- **Static typing** allows the JVM to emit specialised machine code that does not need type-checking branches. A `getfield` instruction becomes a single load from a known offset.
- **Profiling feedback** lets the JIT compiler discover that a virtual method call always goes to the same concrete class; it can then inline that method and guard the inlined code with a cheap type check, a technique known as *devirtualisation*.
- **Threading**: CPython has the Global Interpreter Lock (GIL), which prevents multiple native threads from executing Python bytecode simultaneously. The JVM allows true parallel execution of threads using native OS threads, with memory visibility guarantees defined by the Java Memory Model (JMM).

For CPU-intensive, parallel workloads, Java easily outperforms CPython by a factor of 5–50. However, Java's startup time and memory footprint are larger. A typical "Hello World" Python script runs in milliseconds, while the JVM needs a few hundred milliseconds just to load the runtime. Tools like GraalPy bring the Graal JIT compiler to Python, narrowing the gap, but the JVM's decades of investment in low-pause garbage collectors and lock-elision techniques remain unmatched in the Python world.

1.4 Bytecode and Class Files

When you compile `Hello.java`, the compiler produces one `Hello.class` file per class. This file is a binary container whose format is specified in the *Java Virtual Machine Specification*. The file begins with the magic number `0xCAFEBABE`, followed by the minor and major version numbers (e.g., 61 for Java 17, 65 for Java 21). Next comes the *constant pool*, a table of strings, symbolic references to methods and fields, and numeric constants. After the constant pool come the class's access flags, this class's name, its superclass, implemented interfaces, fields, methods, and attributes.

You can disassemble a class file with `javap -c -p Hello`. The output shows each method's bytecode as a sequence of mnemonic instructions. For example, a simple `return 42;` in an

integer method compiles to `bipush 42` followed by `ireturn`. A `synchronized` block becomes `monitorenter` and `monitorexit` instructions, which lock and unlock the object's intrinsic monitor. A static initialiser is compiled into a special `<clinit>` method that the JVM invokes once when the class is first loaded.

Understanding bytecode is invaluable when diagnosing performance anomalies. For instance, inspecting the bytecode of a method that concatenates strings in a loop will reveal repeated `new StringBuilder` allocations, explaining the memory pressure.

1.5 JIT Compilation

The HotSpot JVM (the most widely used JVM) starts by interpreting bytecode. At the same time, it maintains per-method invocation and loop back-edge counters. When a method's invocation count exceeds a threshold (typically 10,000 calls in server mode), the method is queued for compilation by the JIT compiler.

HotSpot has two JIT compilers:

- **C1 (Client Compiler)**: Produces machine code quickly, performing basic optimisations like inlining of trivial methods and simple register allocation. It collects profiling information (type profiles, branch probabilities) while it runs.
- **C2 (Server Compiler)**: A highly optimising compiler that applies aggressive inlining, escape analysis, lock coarsening, loop unrolling, and vectorisation. Compilation is slower, but the generated code is much faster.

Modern HotSpot uses *tiered compilation*. Level 0 is pure interpretation. Level 1 and 2 use C1 with increasing profiling. Level 3 is C1 with full profiling. Level 4 is C2. A method typically progresses through the tiers as it gets hotter. This gives the best of both worlds: quick warm-up and ultimate peak performance.

The JVM can deoptimise compiled code if an assumption is invalidated. For example, if a method was inlined assuming only one receiver class, and a new class appears, the compiled frame is replaced by an interpreter frame at a safe point, and execution resumes in the interpreter. This *uncommon trap* mechanism enables speculative optimisation without sacrificing correctness.

1.6 GraalVM and Native Image

GraalVM is an alternative JDK that uses a JIT compiler written in Java (Graal). It can replace C2 and offers better peak performance for some workloads due to more advanced inlining heuristics and partial escape analysis. The most transformative component is `native-image`, an ahead-of-time compiler that takes the application's bytecode and produces a single, self-contained executable that does not depend on a JVM at runtime.

The native image build performs a closed-world analysis: it traces all code reachable from the entry point and compiles only that code. The process resolves all reflection, resource loading, and JNI calls that are declared via configuration files or inferred from annotations. The result is an executable that starts in microseconds and has a memory footprint measured in megabytes rather than hundreds of megabytes. This makes Java viable for serverless functions and microservices where cold start time is critical.

The trade-offs: dynamic class loading (like `Class.forName` with unknown names) is not supported unless all possible classes are included in the configuration. Dynamic proxies and bytecode generation at runtime (e.g., CGLIB, Hibernate proxies) must be configured

ahead of time. The peak throughput of native image executables is usually slightly lower than JIT-compiled code because the optimisations that rely on runtime profiling (like devirtualisation and branch prediction based on actual data) are absent.

1.7 Java Version History

Java's release model shifted dramatically in 2017. Before then, feature releases were infrequent. Java 7 (2011) brought invokedynamic and try-with-resources. Java 8 (March 2014) was a tectonic shift: lambda expressions, the Stream API, the new Date and Time API (`java.time`), default methods in interfaces, and the `Optional` class. Most of the industry stayed on Java 8 for years because it represented such a massive change in programming style.

Java 9 (September 2017) introduced the module system (Project Jigsaw), which divided the monolithic standard library into modules, and added `jshell` (an interactive REPL). Java 10 (March 2018) gave us local variable type inference (`var`). Java 11 (September 2018) was the first Long-Term Support (LTS) release after the new six-month cadence. It removed the deprecated Java EE and CORBA modules, standardised `HttpClient`, and Flight Recorder became open source.

Java 17 (September 2021, LTS) was the next "baseline" release. It delivered sealed classes, pattern matching for `instanceof`, enhanced pseudo-random number generators, and the strong encapsulation of internal APIs (JEP 403). Java 21 (September 2023, LTS) is the current reference point for production systems. It introduced virtual threads (Project Loom), record patterns, pattern matching for `switch`, sequenced collections, and string templates (preview). The six-month feature-release train means that preview features are available for experimentation while LTS releases provide a stable, supported foundation.

1.8 Where Java Is Used Today

Java's presence is pervasive. The Spring Boot ecosystem powers millions of microservices, underpinned by Tomcat or Netty. Apache Kafka, the de facto message broker, is written in Java and exposes a Java API. Elasticsearch, the distributed search engine, runs on the JVM. Apache Hadoop and Apache Spark are the backbone of big data, and both are JVM-based. Android's APIs are Java (though the runtime is ART, not HotSpot). Financial trading systems like those built with the Chronicle libraries squeeze microsecond latencies out of the JVM. Build tools (Maven, Gradle), CI/CD servers (Jenkins), and even IDEs (IntelliJ IDEA) are Java applications. Java's reliability, backwards compatibility, and enormous talent pool guarantee its continued use for decades.

Chapter 2

Environment and Tooling

2.1 Installing OpenJDK

The easiest way to manage multiple JDK versions on Unix-like systems is `sdkman` (`https://sdkman.io`). It is a shell script that installs and switches Java distributions. Commands:

- `sdk list java` – shows available versions.
- `sdk install java 21.0.1-open` – installs a specific distribution.
- `sdk use java 21.0.1-open` – switches the current shell session.
- `sdk default java 21.0.1-open` – sets the system default.

On Windows, the official `.msi` installer from adoptium.net or Microsoft's OpenJDK build is the simplest path. After installation, set the `JAVA_HOME` environment variable to the JDK root directory (e.g., `C:\Program Files\Java\jdk-21`). Add `%JAVA_HOME%\bin` to the `PATH` variable. Verify with `java -version` and `javac -version` in a new terminal.

A key detail: on macOS, `/usr/bin/java` is a stub that prompts you to install the JDK. Do not rely on it. Use `sdkman` or download an installer. Always check that `echo $JAVA_HOME` points to the JDK root, not the `/bin` subdirectory.

2.2 The javac Compiler and java Runtime

`javac` transforms `.java` files into `.class` files. Its most important options:

- `-cp <path>` or `-classpath <path>`: A list of directories, JARs, and ZIP files where the compiler looks for referenced classes. The default is the current directory.
- `-d <dir>`: Output directory for `.class` files. This must already exist.
- `-source <version>`: Limit the language features to those of a specific version (e.g., `-source 11` disallows `var`).
- `-target <version>`: Emit class files compatible with an older JVM.
- `-module-path <path>`: For modular applications, replaces `-cp`.
- `-add-exports` and `-add-reads`: Escape hatches for module access.

The `java` command launches the JVM. Its syntax: `java [options] <main-class> [args]`. The `-cp` flag works identically. The `-jar` flag runs an executable JAR whose `META-INF/MANIFEST.MF` declares a `Main-Class` attribute. Starting with Java 9, `java -module <module>/<mainclass>` runs a class from a named module.

Important JVM options (prefixed with `-X` or `-XX`):

- `-Xmx2g` – maximum heap size (e.g., 2 GB).

- `-Xms2g` – initial heap size. Set equal to `-Xmx` for server apps to avoid heap resizing overhead.
- `-XX:+UseG1GC` – enable the G1 garbage collector (default since Java 9, but explicit is fine).
- `-XX:MaxGCPauseMillis=200` – soft target for G1 pause times.

2.3 IntelliJ IDEA vs Eclipse vs VS Code

IntelliJ IDEA: The de facto standard for enterprise Java development. The Community edition is free and covers core Java, Maven/Gradle, and Git. The Ultimate edition adds support for Spring Boot, Jakarta EE, database tools, and advanced profiling. IntelliJ's indexing engine parses the entire project on import, which leads to highly accurate code completion, instant goto-class, and reliable refactorings like "Extract Method" and "Rename" across the entire codebase. Its static analysis is best-in-class, finding many potential NullPointerExceptions and resource leaks. The initial indexing can be slow on very large projects (several minutes), but subsequent interactions are instantaneous.

Eclipse: The original extensible IDE. It has a fast incremental compiler that compiles code on save, even if the rest of the project has errors. This makes it popular in some legacy enterprises. However, Eclipse's workspace concept is brittle; metadata corruption can cause projects to fail to open. Its refactoring and autocompletion, while functional, are less sophisticated than IntelliJ's. The plugin marketplace is vast but of varying quality.

VS Code: With the "Extension Pack for Java" (a bundle of Microsoft and Red Hat extensions), VS Code offers a lightweight, polyglot coding experience. It uses the Eclipse JDT Language Server for Java support, so it gets the same incremental compilation but lacks the deep project-wide analysis that IntelliJ's proprietary indexer provides. Code navigation and refactoring are functional but less reliable, especially for multi-module Maven projects. VS Code shines when you are working in a microservices repo that mixes Java, TypeScript, and Python; you can have a single editor for all languages.

For heavy-duty backend development, IntelliJ Ultimate is the recommended tool. Eclipse survives in some corporate environments due to legacy plugins. VS Code is ideal for quick edits, lightweight services, and polyglot teams.

2.4 Maven

Maven is a declarative build tool. The file `pom.xml` in the project root defines everything. Minimal example:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>myapp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
  </dependencies>
</project>
```

Maven's build lifecycle is a fixed sequence of phases: `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`. Each phase invokes zero or more plugin goals. For instance, the `compile` phase triggers the `maven-compiler-plugin`, and `test` triggers the `maven-surefire-plugin`. The command `mvn install` runs all phases up to `install`, which compiles the code, runs tests, packages the JAR, and installs it into the local repository (`~/.m2`).

Dependency management is transitive: if A depends on B and B depends on C, C is automatically added to A's classpath. Version conflicts are resolved by the "nearest definition" rule (the shortest path from the root). Maven's rigidness is a feature: project layout (`src/main/java`, `src/test/java`) is fixed, making it easy to onboard new developers. However, when build logic deviates from convention, the `pom.xml` can become bloated with plugin configurations. For complex builds, Gradle's programmatic approach may be easier.

2.5 Gradle

Gradle's build scripts (`build.gradle` in Groovy or `build.gradle.kts` in Kotlin) are DSLs that programmatically construct a directed acyclic graph of tasks. A simple Java project:

```
plugins {
    id 'java'
}
dependencies {
    implementation 'com.google.guava:guava:32.0.1-jre'
}
```

Gradle's performance advantage comes from its incremental build model. Each task declares its inputs (e.g., source files) and outputs (e.g., `.class` files). If inputs haven't changed, the task is skipped. The build cache can share outputs across developer machines via a remote HTTP cache. The Gradle wrapper (`gradlew`) is a shell script and a small JAR that download and invoke the correct Gradle version, ensuring reproducible builds.

The flexibility of Gradle is a double-edged sword. Simple builds remain readable, but deeply customized plugins written in Groovy can become imperative spaghetti. For standard Java library or Spring Boot projects, either Maven or Gradle works. Gradle's faster feedback loop is noticeable in large multi-module projects.

2.6 Creating a Runnable JAR

A JAR file is a ZIP archive containing `.class` files and a `META-INF/` directory. To make it executable, the manifest inside `META-INF/MANIFEST.MF` must contain the line `Main-Class: com.example.Main`. The `jar` command can create such a JAR:

```
jar cfe myapp.jar com.example.Main -C build/classes .
```

When dependencies exist, a *fat JAR* (also called uber JAR) bundles all dependency classes into a single archive. For Maven, the `maven-shade-plugin` does this. It copies all classes, and handles overlapping resources by merging or selecting. For Gradle, the `com.github.johnrengelman.sha` plugin provides the `shadowJar` task.

In containerised deployments, a layered JAR is preferred. Spring Boot's Maven and Gradle plugins build a JAR that separates application classes, dependencies, and the loader into distinct layers. Docker layer caching then means that changing only your application code does not require re-uploading the large dependency layer. The executable JAR can still be run with `java -jar`, but its internal structure is optimized for Docker.

Chapter 3

Language Foundations

3.1 Program Structure

A Java source file must follow a strict order: optional `package` declaration, then import statements, then type declarations (classes, interfaces, enums). The package maps to a directory tree; `package com.example.util;` corresponds to the file living in `com/example/util/`. Imports are purely a convenience; you could write `java.util.List<String> list = new java.util.ArrayList<>();` without importing anything. Wildcard imports like `import java.util.*;` bring the immediate members of a package into scope, but if two packages have types with the same name (e.g., `java.util.Date` and `java.sql.Date`), wildcards create ambiguity and compile errors. Always import explicit types.

The file can contain multiple top-level classes, but only one can be declared `public`. That public class's name must match the filename. The entry point method is `public static void main(String[] args)`. The JVM calls this method on the class you specify; the class is not instantiated. Because the method is `static`, you cannot reference instance fields in it unless you create an instance. From Java 21, a preview feature allows writing simply `void main()` (instance method, no arguments), which is useful for teaching and scripting.

3.2 Primitive Types in Full

Java's primitive types are not objects; they are direct machine values. This distinction drives both performance and pitfalls.

- `byte`: 8-bit signed integer, range [-128, 127]. Default 0. Useful for raw binary data (file I/O) and large arrays where memory is critical. Arithmetic on bytes is performed in `int` precision; the result must be explicitly cast back to `byte`.
- `short`: 16-bit signed, range [-32,768, 32,767]. Rarely used; almost always `int` is better unless you have millions of values (e.g., arrays for image processing).
- `int`: 32-bit signed, range approx. -2.147 to 2.147 billion. The default choice for integer arithmetic.
- `long`: 64-bit signed. Append `L` to literals. Used for timestamps, file sizes, and counters that exceed 2 billion.
- `float`: 32-bit IEEE 754 single-precision. Provides about 7 decimal digits of precision. Suffix `f` required. Avoid for money; rounding errors accumulate quickly.
- `double`: 64-bit IEEE 754 double-precision. About 15-16 decimal digits. The default for floating point.
- `char`: 16-bit unsigned, representing a UTF-16 code unit. Range [0, 65535]. Because Unicode characters outside the Basic Multilingual Plane (like many emoji) need two `char`s (a surrogate pair), a single `char` does not always represent a complete character. This is why `String.codePointAt` and `String.codePoints()` exist.

- `boolean`: `true` or `false`. The JVM spec does not define its size; in practice, a `boolean` field usually occupies 1 byte, and in a `boolean[]` array it is typically 1 byte per element. The JVM does not have dedicated boolean instructions; comparisons use integer opcodes.

Precision mistakes are common. The expression `0.1 + 0.2` in double does not exactly equal 0.3; it equals `0.30000000000000004`. This is because 0.1 and 0.2 cannot be represented exactly in binary fractions, just as 1/3 cannot be represented exactly in decimal. Always use `BigDecimal` for precise decimal arithmetic.

3.3 Primitives vs Reference Types

Primitive variables hold the actual value. Reference variables hold a pointer (or handle) to an object on the heap. This distinction has profound effects:

- **Memory layout**: An `int` field sits directly inside the object's memory. An `Integer` field is a reference (4 or 8 bytes depending on compressed OOPs) plus the separate `Integer` object (12–16 bytes of header + 4 bytes for the `int` value, rounded up to a multiple of 8). For large collections, the overhead can multiply memory usage by 4x or more.
- **Nullability**: Primitives can never be `null`. Wrappers can. Autoboxing a `null` reference causes a `NullPointerException`, which is surprising to newcomers.
- **Identity vs equality**: == compares primitive values directly. For reference types, == compares whether two references point to the exact same object. The classic trap: `Integer a = 128; Integer b = 128; System.out.println(a == b);` prints `false` because `Integer.valueOf` caches values from -128 to 127, but 128 is outside that range, so two distinct objects are created. Always use `equals()` to compare object contents.

3.4 Literals

Integer literals are `int` by default. The suffix `L` (or lowercase `l`, which is easy to confuse with 1) makes them `long`. Since Java 7, you can insert underscores for readability: `int billion = 1_000_000_000;`. *Underscores cannot be at the beginning, end, or next to a decimal point.*

Hexadecimal literals start with `0x` (e.g., `0xFF = 255`). Binary literals start with `0b` (e.g., `0b1010 = 10`). Octal literals start with a leading zero (`010 = 8`), but their use is strongly discouraged because the leading zero is often accidental and alters the value.

Floating-point literals default to double. Append `f` to get a float: `3.14f`. Scientific notation is supported: `1.23e-4` (double). Character literals use single quotes: `'A'`. Escape sequences: `\n` (newline), `\t` (tab), `\\` (backslash), `\'` (single quote), `\"` (double quote), `\uXXXX` (Unicode). The null literal is assignable to any reference type.

3.5 Wrapper Classes

Autoboxing is a compiler fiction. When you write `Integer x = 5;`, the compiler emits `Integer x = Integer.valueOf(5);`. Similarly, `int y = x;` becomes `int y = x.intValue();`. This is seamless but introduces subtle overhead:

```
Integer sum = 0;
for (int i = 0; i < 1000000; i++) {
    sum += i; // sum unboxed, addition performed, result boxed again
}
```

This loop creates a million temporary Integer objects, consuming CPU and memory. In performance-sensitive code, prefer int directly. Also, beware that Integer.valueOf caches instances between -128 and 127 by default (the upper bound can be tuned with -XX:AutoBoxCacheMax). Two autoboxed integers within that range are reference-equal, but outside they are not. Never rely on == for wrappers.

3.6 var Keyword (Java 10+)

var instructs the compiler to infer the static type of a local variable from the right-hand side. The bytecode is identical to explicitly declaring the type. var list = new ArrayList<String>(); infers ArrayList<String>. The variable is still strongly typed; you cannot later assign an Integer to it.

Use var when the type is obvious from the right-hand side, especially for long generic types like var entries = new HashMap<String, List<Employee»();. Avoid var when it obscures the type: var result = process(); tells the reader nothing about what result is. Some coding standards (including the official Java style guide) discourage var for primitive types because it makes the code less explicit. A common misperception is that var introduces dynamic typing; it does not. Java's type system remains purely static.

3.7 final Keyword

final on a variable means ‘‘assign once’’. For local variables and method parameters, the variable cannot be reassigned, but the object it points to is still mutable:

```
final List<String> names = new ArrayList<>();
names.add("Alice"); // allowed
names = new ArrayList<>(); // compile error
```

For fields, a final instance field must be assigned by the end of every constructor. A final static field (a constant) must be assigned in a static initializer or directly at declaration. The JVM treats static final fields of primitive or String type as ‘‘compile-time constants’’ if the initializer is a constant expression; the field's value is inlined into other classes at compile time. Changing such a value requires recompiling all dependent classes.

Chapter 4

Operators and Control Flow

4.1 Operators and Precedence

Java has 37 operators. Their precedence (from highest to lowest):

1. Postfix: expr++ expr-
2. Unary: ++expr -expr +expr -expr !
3. Multiplicative: * / %
4. Additive: + -
5. Shift: « » »>
6. Relational: < > <= >= instanceof
7. Equality: == !=
8. Bitwise AND: &
9. Bitwise XOR: ^
10. Bitwise OR: |
11. Logical AND: &&
12. Logical OR: ||
13. Ternary: ? :
14. Assignment: = += -= etc.

When in doubt, parenthesize. The shift operators deserve explanation: « shifts bits left, filling with zeros; this is equivalent to multiplying by powers of two. » shifts right, filling with the sign bit (arithmetic shift). »> shifts right, filling with zero (logical shift), used for unsigned semantics.

Compound assignment operators (+=, -=, *=, /=, %=, «=, »=, »>=, &=, ^=, |=) perform an implicit cast back to the left operand's type. This is why byte b = 0; b += 1; compiles, but b = b + 1; fails because b + 1 promotes to int. This implicit narrowing can hide overflow but is defined by the language.

4.2 Integer Overflow

Java integer arithmetic wraps on overflow following two's complement rules. Integer.MAX_VAL + 1 becomes -2147483648. This is not a bug; it's specified behavior (JLS 4.2.2). However, it's usually unexpected. The methods Math.addExact, subtractExact, multiplyExact, incrementExact, etc., throw ArithmeticException if overflow occurs. They are implemented using bit-level checks and are intrinsified by the JIT to native instructions that detect overflow (e.g., on x86, checking the overflow flag after an add). Use them for critical calculations (e.g., array sizes, financial amounts) unless you deliberately rely on overflow behavior (e.g., hash functions).

4.3 Short-Circuit Evaluation

&& and || evaluate left-to-right, and the right operand is evaluated only if necessary. This is used extensively for null guards:

```
if (obj != null && obj.getValue() > 5) { ... }
```

If obj is null, the right side is never evaluated, so no NullPointerException occurs. The bitwise & and |, when used on boolean operands, always evaluate both sides. They are occasionally used to enforce the evaluation of side-effectful methods, but that pattern is error-prone. Short-circuiting can mask side effects in methods called on the right side; avoid methods with side effects inside conditions.

4.4 Switch Statement vs Switch Expression

The original switch statement (C-style) suffers from fall-through: execution continues into the next case unless a break is encountered. Missing break is a common source of bugs. Java 14 introduced arrow syntax:

```
int numLetters = switch (day) {
    case MONDAY, FRIDAY, SUNDAY -> 6;
    case TUESDAY                -> 7;
    case THURSDAY, SATURDAY     -> 8;
    case WEDNESDAY              -> 9;
};
```

The arrow form does not fall through. Each case label can be a single value, multiple comma-separated values, or a block with yield. The switch expression is exhaustive: if the selector is an enum, all constants must be covered, or a default branch must be present. For sealed class hierarchies (Java 21 pattern matching), the exhaustiveness check extends to the permitted subclasses, enabling the compiler to guarantee that no case is missed.

4.5 Enhanced For Loop

for (String s : list) works on any Iterable. The compiler translates it into:

```
for (Iterator<String> it = list.iterator(); it.hasNext(); ) {
    String s = it.next();
    // body
}
```

For arrays, it generates an indexed loop. During iteration, structural modification of the collection (adding or removing elements via the collection's methods, not the iterator's) causes a ConcurrentModificationException on the next hasNext() or next() call. The exception is a best-effort mechanism; it does not guarantee detection in all concurrent scenarios. To remove elements while iterating, use the iterator's remove() method, or use removeIf() on the collection, which is the preferred approach in Java 8+:

```
names.removeIf(name -> name.startsWith("A"));
```

4.6 Labeled break and continue

Labels allow you to break out of or continue an outer loop from a nested context. The syntax is labelName: for(...) and then break labelName; or continue labelName;. Example:

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (i * j > 3) break outer;
    }
}
```

The break outer transfers control to the statement immediately after the labeled loop. This is a structured goto that is more readable than setting boolean flags. That said, if the nested loop is complex enough to require a label, consider extracting the inner loop into a separate method and using a return instead. Labels are legal only on loops and blocks; they cannot be used to jump arbitrarily.

4.7 Ternary Operator

condition ? a : b selects a if condition is true, else b. The types of a and b must be assignment-compatible. If one is a primitive and the other a wrapper, autounboxing may throw NullPointerException. For example: Integer x = null; int y = flag ? x : 0; will unbox x and throw NPE. The ternary operator is right-associative, so a ? b : c ? d : e evaluates as a ? b : (c ? d : e). Avoid nesting ternaries more than one level deep; it rapidly becomes unreadable.

Chapter 5

Methods

5.1 Method Signatures

The signature of a method for the purposes of overloading is its name plus the ordered list of parameter types. The return type, access modifier, and throws clause are not part of the signature. For overriding, the signature must match exactly; a difference in return type is allowed only if it is a covariant return (subtype). A method's descriptor in the bytecode includes the return type and parameter types encoded as a single string, but the Java language treats overloading resolution differently.

5.2 Pass-by-Value

Every argument in Java is passed by value: a copy of the actual parameter is made. For primitives, the copy is of the value itself. For reference types, the copy is of the reference. This means:

- Changing the value of a primitive parameter has no effect on the caller.
- Reassigning the reference parameter (e.g., param = new Something();) does not affect the caller's reference.
- Mutating the object that the reference points to (e.g., param.setField(...)) does affect the caller's object, because both the caller's reference and the parameter copy point to the same object.

Many developers incorrectly believe Java passes objects by reference. This misunderstanding leads to attempts to write a generic swap method, which cannot work:

```
public static void swap(Object a, Object b) {
    Object temp = a;
    a = b;
    b = temp; // only local copies change
}
```

5.3 Method Overloading

Overloading allows multiple methods with the same name but different parameter lists. The compiler resolves the call in three phases:

1. Find methods applicable by strict invocation (exact types or unboxing). 2. If not found, apply widening primitive conversions. 3. If still not found, allow boxing and then widening.

Ambiguities arise when overloaded methods have signatures like void m(int a, long b) and void m(Integer a, Integer b). A call m(1, 1) will prefer the first because widening is tried before boxing. Avoid overloading methods with the same

number of parameters that differ only in wrapper vs primitive. For constructors, prefer static factory methods with descriptive names instead of overloading.

5.4 Varargs

Varargs are compiled into an array parameter. The method void print(String... args) is equivalent to void print(String[] args) at the bytecode level, but the caller can pass zero or more arguments separated by commas. The compiler creates an array at the call site. This array allocation overhead is negligible in most cases, but for low-level utilities called in hot loops, it's common to see manual overloads for 0, 1, 2, and 3 arguments before a varargs fallback, as the JDK does in EnumSet.of.

A single method cannot have two varargs parameters, and the varargs parameter must be the last. A potentially ambiguous situation arises with void m(String... args) and void m(String first, String... rest); a call with a single argument matches both. The compiler will prefer the non-varargs overload (String first, String... rest) only if the first argument is given; otherwise, the pure varargs version is chosen. To avoid confusion, either design the API without such overloads, or document the resolution.

5.5 Static Methods vs Instance Methods

A static method is associated with the class, not an instance. It can access only static fields and static methods. It cannot use this or super. Because it is resolved at compile time based on the reference type, static methods are not polymorphic. They can be shadowed in subclasses, but a call through a superclass reference will always invoke the superclass version.

Use static methods for pure functions that depend only on their arguments: Math.sqrt, Collections.sort, utility methods. A common anti-pattern is to avoid static methods because they ‘‘can't be mocked in tests.'' Modern mocking frameworks like Mockito can mock static methods via mockStatic. For truly stateless utilities, static methods are the correct design.

Instance methods support dynamic dispatch: the actual method called depends on the runtime class of the object, determined by the virtual method table. This is the foundation of polymorphism.

5.6 Recursion

Recursion is a natural fit for problems defined recursively, such as tree and graph traversals. Each recursive call consumes a stack frame, which holds the method's parameters, local variables, and return address. The default thread stack size is 1 MB on most JVMs. This allows roughly 10,000-30,000 frames, depending on frame size. Exceeding this causes a StackOverflowError, which is an Error and should not be caught.

Java does not eliminate tail calls. Even if the recursive call is the last operation, a new frame is always allocated. For deep iterations, an iterative approach using a loop or an explicit stack (e.g., Deque) is safer and often faster, as it avoids method call overhead. For tree structures with bounded depth (e.g.,

a balanced binary tree), recursion is acceptable. Always ensure a base case is reachable and test with worst-case input sizes.

Chapter 6

Arrays

6.1 Array Declaration, Initialization, Memory Layout

In Java, an array is an object that holds a fixed number of values of a single type. Despite being objects, arrays do not have a corresponding class visible in the standard library; the JVM creates array classes dynamically at runtime. An array's length is established at creation and can never change; the length is stored in the length field (not a method) of the array object.

Declaration syntax: Java offers two declaration styles, but only one is idiomatic. The C-style int numbers[] compiles but is discouraged because it places the brackets next to the variable name rather than the type. The preferred form is int[] numbers - ''int array named numbers''. This reads naturally and avoids confusion when declaring multiple variables on one line: int[] a, b; declares two arrays, while int a[], b; declares one array and one int.

Initialization: An array can be initialized with an array initializer, a braced list of values:

```
int[] primes = {2, 3, 5, 7, 11};  // only in declarations
String[] names = new String[]{"Alice", "Bob"}; // anywhere
```

The initializer syntax can only be used in a declaration. Otherwise, you must use new with an explicit array creation expression or use a factory method.

When you allocate an array with new int[10], the JVM allocates a contiguous block of memory on the heap that holds exactly ten 4-byte integers, all initialized to their default value (0). For new Object[5], the array stores five references (4 or 8 bytes each, depending on compressed OOPs), all initialized to null. The array object itself has a standard object header (mark word + klass pointer) plus an int for the length. This overhead is typically 12-16 bytes, so even an empty array consumes some memory.

Bounds checking: Every array access is preceded by a runtime bounds check that the index is in [0, length). If the check fails, an ArrayIndexOutOfBoundsException is thrown. This check adds a small cost, but the JIT compiler can often eliminate it when it can prove the index is always within bounds (e.g., in a simple for loop).

Common mistake: Assuming that array elements are automatically initialized to something meaningful. Primitive elements default to 0, 0.0, '\u0000', or false. Reference elements default to null. Forgetting to assign values before reading can lead to NullPointerException or incorrect calculations.

Production best practice: Declare arrays as final fields if the array reference should not change, but remember that the array content is still mutable. To expose an array from an API without allowing external mutation, either return a copy via Arrays.copyOf or wrap it in an unmodifiable list. Do not return the internal array reference directly because callers can corrupt your state.

6.2 Multi-dimensional Arrays

Java implements multidimensional arrays as arrays of arrays, a model sometimes called "ragged" or "jagged" arrays. The syntax `int[][] matrix = new int[3][4];` is shorthand for: create an array of length 3, then for each slot create an `int[4]`. There is no requirement that sub-arrays have the same length; you can create triangular structures.

Memory layout: The top-level array holds references to the row arrays. Each row array is a separate contiguous block on the heap. Because the rows are independent objects, memory access patterns matter. Iterating over `matrix[row][col]` with the outer loop over rows and inner over columns is cache-friendly because each row's elements are contiguous. Reversing the loop order (column-wise traversal) causes a cache miss for each access because the next element is in a different row object, potentially far away in memory.

```
int[][] matrix = new int[3][4];
// Cache-friendly: row-major traversal
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        matrix[i][j] = i * j;
    }
}
```

Jagged arrays: Useful when each row has a different number of elements, such as storing a triangular matrix or a list of varying-length lines.

```
int[][] triangle = new int[3][];
triangle[0] = new int[]{1};
triangle[1] = new int[]{2, 3};
triangle[2] = new int[]{4, 5, 6};
```

Common mistake: Attempting to use a true rectangular allocation syntax like `new int[3,4]`, which does not exist. The only way is `new int[3][4]`. Also, forgetting to initialize the sub-arrays before accessing elements leads to `NullPointerException`.

Performance: Multidimensional arrays in Java carry the overhead of multiple object references and bounds checks. For numerical computing with large dense matrices, consider using a single flat `double[]` and manual index calculations (`row * cols + col`). This eliminates the indirection and improves cache locality and SIMD optimization opportunities by the JIT.

6.3 System.arraycopy vs Arrays.copyOf vs clone

Copying arrays is a frequent operation, and Java provides several mechanisms with different tradeoffs.

`System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length)`: This is a native method implemented in C within the JVM. It performs a direct memory copy of the array region using highly optimized routines (often leveraging SIMD instructions). It can copy between arrays of the same type or compatible reference types (with a runtime check). It throws `IndexOutOfBoundsException` or `ArrayStoreException` if the destination cannot hold the source elements (e.g., copying `String[]` into an `Integer[]`). It does *not* allocate a new array; you must pre-allocate the destination. This method is the fastest bulk copy primitive.

Arrays.copyOf(T[] original, int newLength): This convenience method creates a new array of the specified length and internally uses System.arraycopy to copy elements from the original. If newLength is larger than the original, the extra elements are set to defaults (zero or null). If shorter, the result is a truncated copy. It is type-safe and returns an array of the same component type as the original.

Object.clone() on arrays: An array's clone() method returns a new array with the same length and elements. The copy is shallow: for arrays of primitives, the values are copied; for arrays of references, the references are copied, but the objects they refer to are not cloned. The returned type is Object, requiring an explicit cast, e.g., int[] copy = (int[]) original.clone();. The cast is safe but clutters the code. clone() is slightly slower than System.arraycopy because it must also create the new array and set the length, but the difference is minimal for one-off copies.

```
int[] original = {1, 2, 3, 4, 5};
// Using arraycopy
int[] dest1 = new int[original.length];
System.arraycopy(original, 0, dest1, 0, original.length);
// Using copyOf
int[] dest2 = Arrays.copyOf(original, original.length);
// Using clone
int[] dest3 = original.clone();
```

Recommendation: Use Arrays.copyOf for clarity and type safety when you need a new array of the same or different length. Use System.arraycopy when you are copying into an existing array to avoid allocation overhead, e.g., in manual resizing or ring buffers. Avoid clone() on arrays for new code because of the cast requirement and its shallow-copy semantics that are less obvious.

Common mistake: Assuming clone() performs a deep copy. For reference arrays, clone() copies the references, so the original and the clone share the same element objects. Mutating an object through one array will affect the other. To deep copy, iterate and explicitly copy each element.

6.4 Arrays Utility

The java.util.Arrays class contains static utility methods that operate on arrays.

sort: For primitive arrays, it uses a dual-pivot quicksort (by Vladimir Yaroslavskiy) that is O(n log n) on average and very fast due to cache efficiency. For object arrays, it uses TimSort, a stable adaptive merge sort that performs well on partially sorted data. TimSort is O(n log n) worst-case and preserves the relative order of equal elements. There are overloads that accept a Comparator. Sorting is in-place.

binarySearch: Searches a sorted array in O(log n) time. The array must be sorted according to the same ordering used for the search, or the result is undefined. Returns the index if found, or -(insertion point) - 1 if not found. This allows you to compute the insertion point.

fill: Sets every element of an array (or a range) to a specified value. Useful for reinitializing arrays.

asList(T... a): Returns a fixed-size list backed by the array. Changes to the list write through to the array, and vice-versa. The list is not an ArrayList but a private static inner class of Arrays. It does not support add, remove, or any structural modification; attempting these throws UnsupportedOperationException. This method is often misused to create a list that later needs to grow. Use new

ArrayList<>(Arrays.asList(...)) to get a fully modifiable list. Note: for a single element, the varargs can be misleading; Arrays.asList(1) returns a List<int[]> if called with an int[] argument because the array itself is autoboxed? Actually, Arrays.asList(int[]) will treat the int[] as a single object (since T is int[]), resulting in a list with one element: the array. This is a common gotcha.

```
int[] arr = {1,2,3};
List<int[]> list = Arrays.asList(arr); // size 1!
// Correct way for primitives: use stream boxing
List<Integer> boxedList =
    Arrays.stream(arr).boxed().collect(Collectors.toList());
```

toString and deepToString: toString returns a readable representation like [1, 2, 3]. For multidimensional arrays, use deepToString to recursively convert sub-arrays. Calling toString on an int[][] will print the addresses of the row arrays, not the elements.

equals and deepEquals: Compare arrays element-wise. equals works for one-dimensional arrays; deepEquals recursively compares nested arrays. The Object.equals method on arrays just checks reference identity, so arr1.equals(arr2) is true only if they are the same object.

hashCode and deepHashCode: Compute a hash code based on the array contents, consistent with equals and deepEquals.

copyOfRange: Similar to copyOf but copies a specified subrange.

mismatch: (Java 9) Returns the index of the first mismatch between two arrays, or -1 if they are identical.

Production tip: Always use Arrays.toString for logging array contents. If you see [I@15db9742 in a log, you forgot to call Arrays.toString.

6.5 Array vs ArrayList

The choice between an array and an ArrayList involves tradeoffs in performance, flexibility, and API compatibility.

When to use an array:

- The size is fixed and known at compile time (e.g., days of the week, chess board).
- You need the absolute best performance in a hot loop and want to avoid method call overhead and bounds-check duplication (though the JIT often eliminates bounds checks in simple loops anyway).
- You are dealing with primitive types and want to avoid boxing/unboxing overhead. An int[] stores raw 4-byte values, while ArrayList<Integer> involves Integer objects (12-16 bytes each plus reference overhead), increasing memory usage significantly.
- You are interacting with APIs that require arrays (e.g., Arrays.sort, System.arraycopy, native methods, or legacy libraries).

When to use ArrayList:

- The size of the collection can change dynamically.
- You need the rich API of the Collections Framework: insertion, removal, iteration, streams, bulk operations.

- You want type-safe generics (arrays are covariant, which can lead to runtime ArrayStoreException if you're not careful).
- You need to integrate with other collections, pass the list to methods expecting List, or serialize it.

Performance comparison: Indexed access on ArrayList is essentially identical to an array because it's just a range-checked access to the backing array. The overhead of the method call (get()) is usually inlined by the JIT. However, ArrayList adds a small memory overhead for the list object itself and the size field, plus the backing array may be larger than the number of elements (capacity vs. size). Iterating a ArrayList via for-each loop is as fast as iterating an array (the JIT unrolls and compiles to direct access).

Memory: An int[] with 1,000 elements occupies roughly 4 KB for the values plus 16 bytes for the array header. An ArrayList<Integer> with 1,000 elements holds a backing Object[] of at least 1,000 references (4 or 8 KB), plus 1,000 Integer objects (each 16 bytes header + 4 bytes value = 20 bytes, rounded to 24 bytes for alignment), totaling 28 KB, a 7x increase.

Common mistake: Storing a large amount of primitives in an ArrayList<Integer> without realizing the memory blow-up. For huge collections of primitives, use third-party libraries like Eclipse Collections, fastutil, or Trove that provide object-based containers with primitive-friendly APIs.

When to use immutable lists: If the content is fixed, prefer List.of(1,2,3) (Java 9+). It returns a compact immutable object with no resizing overhead, and it is safe to share.

Summary: Use arrays for low-level performance, primitive data, and fixed-size structures. Use ArrayList for general-purpose dynamic collections and when you need the Collections API.

Chapter 7

Strings in Depth

7.1 String Immutability

String in Java is immutable: once a String object is constructed, its character sequence can never be altered. This is not an accident; it is a design decision that brings several critical benefits at the cost of some allocation pressure.

Why immutability?

1. Security: Strings are used extensively in security-sensitive contexts - class names in class loading, file paths, database URLs, and network connection endpoints. If a String were mutable, a malicious thread could change the path after a security check but before the file was opened, bypassing access control. Immutability makes these attacks impossible.

2. Thread safety: Immutable objects are inherently safe to share across threads without synchronisation. No lock is required to read a String field from multiple threads because its value can never change.

3. Hash code caching: String caches its hash code in a private int hash field. The first call to hashCode() computes the hash and stores it; subsequent calls return the cached value instantly. This works only because the content never changes. Since String is the most common key in HashMap, this caching is a massive performance win.

4. String pool: The JVM maintains an internal pool of unique strings, called the string interning pool. Only compile-time constant strings and those explicitly interned via intern() are placed in the pool. Because strings are immutable, the JVM can safely share one instance among many references, reducing memory footprint.

How immutability is enforced: The String class is final, so no subclass can override its methods to introduce mutable behaviour. All fields are private final. The character data itself is stored in a byte[] (since Java 9, for compact strings) or a char[] (pre-Java 9) that is never exposed to the outside. The constructor makes a defensive copy of any array passed in, and no method returns a reference to the internal array. Every ‘‘modifying’’ method - concat, substring, toUpperCase - allocates a new String object.

The string pool in detail: The pool is stored in the JVM’s runtime constant pool in the Metaspace (or PermGen in older JVMs). When the compiler sees a string literal, it places the string into the class file’s constant pool. At class loading, the JVM interns that string, meaning it checks the pool: if the string already exists, it reuses the existing instance; otherwise, it creates a new one and adds it to the pool. Thus, "hello" == "hello" is true because both references point to the same interned string.

The intern() method explicitly adds any string to the pool. However, manually interning dynamic strings is rarely a good idea. The pool has a fixed memory budget (the Metaspace size), and over-interning can fill it, causing an OutOfMemoryError

that is difficult to diagnose. Moreover, intern() is expensive because it has to look up a hash table; the benefit is only worthwhile if you have a huge number of duplicate strings - a situation better handled by a HashMap<String, String> with weak references.

Common mistake: Assuming that concatenating two interned strings produces an interned string. It does not. The result of "hello" + " world" is interned only if the entire concatenation is a constant expression (both operands are compile-time constants). Dynamic concatenation (involving variables) always creates a new, non-interned string. Calling intern() on such a result will return the canonical instance, but the original non-interned string is still created and later garbage collected, which may be wasteful.

7.2 String Comparison

The == operator compares reference identity. For strings, this means it checks whether two references point to the *same object* on the heap, not whether they contain the same characters. The correct way to compare string content is the equals(Object) method. String.equals overrides Object.equals and performs a character-by-character comparison, with an early exit if the objects are identical or lengths differ.

The classic pitfall:

```
String a = "hello";
String b = new String("hello");
System.out.println(a == b);      // false (different objects)
System.out.println(a.equals(b)); // true  (same content)
```

Even experienced developers get tripped up when comparing strings that come from different sources - e.g., a literal against a user-input string read from a file.

Interning and identity: Strings that are compile-time constant expressions are guaranteed to be interned. Thus, "hel" + "lo" == "hello" is true. But if any operand is a variable or a method call, the compiler cannot determine the value at compile time, so a new string is created. Relying on == for anything but interned constants (e.g., enum-like string constants defined as static final String) is a bug.

Case-insensitive comparison: equalsIgnoreCase(String) performs a character-by-character comparison where upper- and lower-case letters are treated as equal. However, it is locale-agnostic; it uses the rules of the English alphabet. For proper locale-sensiti comparison (e.g., Turkish dotless i), use java.text.Collator. Collator.PRIMARY treats accented characters as identical, whereas equalsIgnoreCase does not.

The null problem: Calling equals on a null reference throws NullPointerException. If there is any chance the reference may be null, use Objects.equals(a, b) (which handles nulls) or write "constant".equals(variable) to ensure the known non-null object receives the call.

7.3 Key Methods

charAt(int index) returns the char (UTF-16 code unit) at the given index. For Unicode characters that lie outside the Basic Multilingual Plane (codepoints U+10000 and above, e.g., many emoji), a single char is not enough - they are represented by a surrogate pair (two consecutive char values). Use codePointAt(index) and

codePointCount() for proper Unicode handling. The common mistake is looping through a string with charAt and assuming each iteration advances by one complete character; for supplementary characters, you need to skip the second surrogate.

substring(int beginIndex, int endIndex) returns a new string that is a copy of the specified range. Before Java 7 update 6, substring shared the underlying char[] with the original string, which could cause memory leaks if a tiny substring was kept while the huge original string was expected to be garbage collected. Since then, substring always makes a full copy, so it is safe but allocates memory proportional to the substring length.

indexOf(int ch) and lastIndexOf(int ch) search for a character and return its index or -1 if not found. Overloads accept a String or a starting index. These methods use a naive linear scan; if you need to search massive strings repeatedly, consider using a more sophisticated algorithm (e.g., Knuth-Morris-Pratt) implemented manually, but for typical use the JIT-compiled indexOf with SSE instructions is extremely fast.

replace(CharSequence target, CharSequence replacement) replaces all non-overlapping occurrences. It compiles the target into a regex only if the target is a String that is used as a pattern? Actually, replace(CharSequence, CharSequence) treats the target literally; it does not use regex. The replaceAll and replaceFirst methods work with regex. Be careful not to confuse them.

split(String regex) splits the string around matches of the given regular expression. The regex can be complex, and special characters like . or | have regex meaning; to split by a literal dot, use Pattern.quote(".") or "\\.". The result array may have trailing empty strings if a limit is not given; to discard them, use split(regex, 0) (the default) which discards trailing empty strings. If you want to keep them, pass a negative limit.

trim() removes whitespace as defined by <= '\u0020' (i.e., ASCII space and lower control characters). strip() (added in Java 11) uses Character.isWhitespace() and thus removes all Unicode space characters, including non-breaking spaces, and also strips leading and trailing zero-width characters. For modern applications dealing with international text, always use strip().

Text transformations: toLowerCase(Locale) and toUpperCase(Locale) require a locale for correct behaviour with certain languages (Turkish, Lithuanian). Use the no-argument forms only when the default locale is intended, which is rarely the case in server-side code. Specify Locale.ROOT for locale-insensitive transformations.

7.4 StringBuilder

Because String is immutable, concatenating many strings in a loop creates a cascade of temporary objects. Each + on strings produces a new string. In a loop with 10,000 iterations, the first iteration creates a string of length 1, the second of length 2, ... resulting in $O(n^2)$ time and massive allocation. StringBuilder (and StringBuffer) are mutable sequences of characters that avoid this waste.

Internal mechanism: StringBuilder maintains a resizable byte[] (or char[]) called the value buffer. When you append, it writes characters into the buffer, expanding it if necessary (copying to a new array of double the size, or less if the required capacity is small). The toString() method creates a new String copying the current buffer. The expansion cost can be mitigated by pre-allocating the approximate final size with the constructor new StringBuilder(expectedLength).

Compiler optimization: Simple string concatenation in a single expression (like

"a" + "b" + c) is translated by the compiler into a StringBuilder chain. Thus, you don't need to manually create a StringBuilder for straightforward concatenations. However, in loops, the compiler does *not* hoist the StringBuilder creation; each iteration would create a new builder, defeating the purpose. That's why you must explicitly declare the builder outside the loop.

```
// Bad: creates a new StringBuilder each iteration
String result = "";
for (String s : list) {
    result += s;  // desugars to: result = new
       StringBuilder(result).append(s).toString();
}
// Good:
StringBuilder sb = new StringBuilder();
for (String s : list) {
    sb.append(s);
}
String result = sb.toString();
```

Capacity management: You can set the capacity with ensureCapacity(minimumCapacity). If you know you will append about 1 million characters, allocating that capacity upfront avoids multiple array copy operations. The default initial capacity is 16 characters. Growth policy: new capacity = old capacity * 2 + 2 (unless the required size is larger, then it becomes that size). The capacity is available via capacity(), which differs from length() (the number of characters currently stored).

Common mistake: Using StringBuilder in a multithreaded context without synchronization. Even though StringBuilder is not thread-safe, code sometimes assumes that because each thread has its own builder it's fine - that's true only if builders are truly not shared. Accidentally sharing a builder (e.g., stored in a field) causes corrupted output due to unsynchronised writes to the same buffer. For thread-safe operations, StringBuffer is a synchronised alternative but rarely needed; local builders per thread are the norm.

7.5 StringBuffer vs StringBuilder

StringBuffer was introduced in Java 1.0 as the only mutable string type. All its public methods are synchronized, which means they acquire the intrinsic lock on the StringBuffer object. In a single-threaded program (or when a buffer is confined to a single thread), this locking overhead is pure waste. StringBuilder (Java 5) removes the synchronization, making its methods faster.

Performance difference: A microbenchmark of appending characters in a tight loop shows StringBuilder to be roughly 30-50% faster than StringBuffer because it avoids monitor enter/exit and the associated memory barriers. The gap widens when the operation is light (e.g., single-character append) because the lock becomes the dominant cost.

When is StringBuffer justified?: Almost never. Even in concurrent environments, it is extremely rare for multiple threads to be appending to the same logical string. If such a shared mutable state is genuinely required, the granularity of StringBuffer is usually wrong - you would want higher-level coordination. It remains in the JDK for backward compatibility only. Use StringBuilder for all new code.

7.6 Text Blocks (Java 15+)

Text blocks are a syntax for multi-line string literals, delimited by three double quotes ("""). They were introduced to stop the pain of embedding large strings (JSON, SQL, HTML) in Java code with manual \n and + concatenation.

The incidental white space algorithm: At compile time, the compiler processes the text block and determines the minimum indentation across all lines. That amount of leading white space is stripped from every line. This means you can indent the text block naturally to match your code indentation, and the resulting string will have only the relative indentation. Trailing white space on each line is stripped unless you explicitly add the \s escape sequence at the end of a line, which tells the compiler to keep trailing spaces.

Line terminators: The newline separator in text blocks is always \n (\u000A), regardless of the platform. If you want platform-specific line separators, use String::replace to substitute. You can place \ at the end of a line to indicate continuation without a newline.

Integration with String::formatted: Text blocks work nicely with formatted() to embed values:

```
String query = """
    SELECT id, name
    FROM users
    WHERE age > %d
    """.formatted(minAge);
```

Common mistake: Forgetting that the opening """ position matters. If the opening triple quotes are on the same line as some text, that first line's indentation is used for calculating the minimum. Usually, you put the opening """ on a new line to avoid extra leading spaces. Also, adding a trailing newline is natural; if you don't want it, end with """. Be aware that the closing triple quotes must be on their own line or at the end of the last line.

7.7 String.format and printf

String.format (and System.out.printf) use the java.util.Formatter engine. The syntax for format specifiers: %[argument_index$][flags][width][.precision]conversion. The most common conversions: %s (string), %d (decimal integer), %f (decimal float), %n (platform line separator). Precision for floats controls digits after the decimal point. Width pads with spaces (right-aligned by default; use - for left alignment).

The %n specifier emits the platform-specific line separator: \r\n on Windows, \n on Unix. Using \n directly works but may cause issues with some Windows tools.

Locale awareness: String.format uses the JVM's default locale. To produce locale-independent output (e.g., for logs or network protocols), use String.format(Locale.RO format, args). This ensures numbers are formatted with a period as decimal separator and no thousand grouping.

Argument indexing: You can reorder arguments with %1$s, %2$d etc. This is useful for internationalization. Less well-known: %<s re-uses the previous argument.

Performance: String.format is considerably slower than StringBuilder because it parses the format string every call. If the same format is used repeatedly, consider using a pre-compiled MessageFormat or manual builder. For occasional use (logging statements that may be disabled), the overhead is acceptable.

7.8 Regular Expressions with Pattern and Matcher

The java.util.regex package provides a Perl-like regex engine. The key classes are Pattern (a compiled regex) and Matcher (the engine that applies the pattern to a specific input).

Compilation cost: Pattern.compile(regex) builds a finite-state automaton. This operation can be expensive (especially for complex regexes with many alternations or quantifiers). Always compile a Pattern once (as a static final field) rather than using String.matches() or String.split() repeatedly with the same regex, because those methods re-compile every time.

Matcher usage: The Matcher is stateful and not thread-safe. The typical flow: Pattern p = Pattern.compile("..."); Matcher m = p.matcher(input); while (m.find()) { String group = m.group(1); }. The find() method finds the next match; matches() requires the entire input to match. lookingAt() matches only at the beginning.

Groups: Parentheses () define capturing groups, numbered by the position of their opening parenthesis, starting at 1. Group 0 is the entire match. Named groups use the syntax (?<name>...) and can be retrieved with m.group("name"). Note: named groups are still internally numbered; they are just aliases.

Catastrophic backtracking: A regex like (a+)+b on input "aaaaaaaaaaaaaaaaaaaaaaaaX" can cause the engine to explore an exponential number of possible matches, freezing the application. The Java regex engine uses a recursive backtracking implementation that is vulnerable to this. Avoid nested quantifiers and be cautious with alternation inside groups. For simple tokenising, consider using a scanner or manual parsing.

Flags: Pattern.CASE_INSENSITIVE (embedded flag (?i)), Pattern.MULTILINE ((?m) makes ^ and $ match start/end of each line), Pattern.DOTALL ((?s) makes . match line terminators). Use flags in Pattern.compile(String, int).

Common mistake: Forgetting to escape the backslash in the Java string literal. The regex \d must be written as "\\d" because \ is the escape character in Java strings. This leads to cryptic compile errors about illegal escape characters.

Chapter 8

Object-Oriented Programming

8.1 Classes and Objects

A class is a template that defines the state (fields) and behaviour (methods) of its instances. When you use the new keyword, the JVM allocates memory on the heap for the object, initialises its header, and invokes the constructor. The memory layout of an ordinary object consists of:

- Mark word (4 or 8 bytes, depending on JVM bitness): contains identity hash code, garbage collector flags, lock information.
- Klass pointer (4 or 8 bytes with compressed OOPs, 8 otherwise): points to the class's internal metadata (the Class object's VM representation).
- Fields in declaration order, with alignment gaps for non-static fields.

Objects are accessed exclusively through references. A reference variable stores a pointer to the object, not the object data itself. Assigning one reference to another copies the pointer, so both point to the same object. This is fundamental to understanding side effects: passing an object to a method does not copy the object; the method receives a copy of the reference and can mutate the object.

8.2 Constructors

A constructor's job is to establish the invariant of the newly created object. Java enforces a call to a superclass constructor (explicitly with super() or implicitly) before the subclass constructor body executes. This guarantees that the superclass part of the object is fully initialised before the subclass adds its own initialisation.

Constructor chaining: You can call another constructor of the same class with this(...). This call must be the first statement. It is commonly used to provide default values for parameters.

Default constructor: If no constructor is declared, the compiler adds a public no-argument constructor. However, if any constructor is explicitly declared, the default is not generated. This causes issues when a framework (like JPA or Jackson) expects a no-arg constructor, and you forgot to add one.

Calling overridable methods: Never invoke a non-final, non-private instance method from a constructor. Because the superclass constructor runs before the subclass constructor, if that method is overridden in the subclass, the overridden version will be called, but the subclass's fields will not yet have been initialised (they are still zero/null). This leads to unpredictable NullPointerExceptions or inconsistent state. The JVM allows it, but it's a well-known anti-pattern.

Compact constructors in records: A record's canonical constructor can be written without parameter list or parentheses; this is called a compact constructor. Its purpose is to validate the state. The compiler copies the component fields after the compact constructor executes. Example:

```
public record Range(int lo, int hi) {
    public Range {  // compact constructor
        if (lo > hi) throw new IllegalArgumentException("lo > hi");
    }
}
```

8.3 Access Modifiers

Access control in Java is based on classes and packages. The four levels:

- public: accessible from any other class.
- protected: accessible within the same package and in subclasses (even if they are in a different package). Note: subclasses can access a protected member only through a reference of their own type, not through a superclass reference (JLS 6.6.2).
- package-private (no modifier): accessible only within the same package. This is the default level and is heavily used for encapsulating implementation details across related classes.
- private: accessible only within the same class (and inner classes/records within that class).

The package is the unit of encapsulation. Carefully choosing which classes share a package is a design activity. Placing too many classes in a package weakens encapsulation; splitting packages too finely complicates visibility. Modules (Java 9+) add another layer of encapsulation at the module level.

8.4 Encapsulation

Encapsulation is not about simply making fields private and providing getters/setters. That is an anti-pattern if done blindly. True encapsulation means hiding internal representation and exposing behaviour, not raw data. Ask ‘‘what does this object do?’’ rather than ‘‘what data does it have?’’.

For example, a BankAccount class should expose deposit(amount) and withdraw(amount) methods that validate the amount and enforce business rules, not setBalance(double).

Benefits: Encapsulation allows you to change the internal representation (e.g., switch from storing balance as a double to a BigDecimal) without breaking clients. It centralises validation logic. It makes the code self-documenting because the public methods clearly state the available operations.

8.5 static Fields and Methods

static fields are class variables; there is exactly one copy per class, regardless of how many instances exist. They are stored in the class’s internal metadata (the Klass structure) in Metaspace, not on the heap. They are initialised when the class is loaded, in the order they appear in the source file, after the superclass’s static initialisation.

The static keyword on a method means it is a class method. It can be invoked without an instance. It cannot use this or super and can only directly access other static members. Because static methods are resolved at compile time based on the declared type of the reference, they are not polymorphic.

Singleton pattern problem: The classic Singleton uses a static field to hold the single instance. This introduces global mutable state, making unit testing difficult (tests cannot run in isolation) and hiding dependencies. Modern practice favors dependency injection: create a single instance and pass it to those that need it, rather than having classes fetch the singleton statically.

8.6 Inheritance

Inheritance lets a class reuse and extend the behaviour of a parent class using the extends keyword. Java's single-inheritance model avoids the ''diamond problem'' where a class inherits from two classes that have conflicting implementations of the same method. Interfaces provide a restricted form of multiple inheritance of type and (with default methods) behaviour.

The subclass constructor chain always calls the superclass constructor. The superclass constructor runs first, setting up the superclass part of the object. This ensures that the superclass invariants hold before the subclass adds its own.

The fragile base class problem: Inheriting from a class couples the subclass to the implementation details of the superclass. Changes in the superclass (adding a new method, modifying the behaviour of a method that the subclass relies on) can break subclasses. Joshua Bloch's advice is to ''prefer composition over inheritance.'' Use inheritance only when there is a true ''is-a'' relationship and the behaviour of the superclass is unlikely to change in ways that would violate expectations.

8.7 Method Overriding

When a subclass provides a different implementation of a method defined in a superclass, we say it overrides that method. The rules:

- The parameter list must be exactly the same (types, order, count).
- The return type must be the same or a subtype (covariant return).
- The access modifier cannot be more restrictive (e.g., cannot change public to protected).
- The overriding method may throw fewer, narrower, or no checked exceptions; it cannot throw broader exceptions.

The @Override annotation is not merely documentation - it tells the compiler ''I intend to override a method.'' If there is a typo in the method name or parameter types, the compiler produces an error. Always use it.

The JVM implements virtual method dispatch via a *vtable* (virtual method table). Each class has a vtable containing pointers to the actual method implementations for each virtual method. When you call a method on an object, the JVM follows the klass pointer to the class's vtable and calls the method at the correct offset. The JIT can often inline virtual calls through profiling.

8.8 super Keyword

super has two uses:

- super(args) - invokes the matching superclass constructor. This must be the first statement in the constructor body. If you don't write it, the compiler inserts super() (calling the no-arg superclass constructor), which fails if the superclass lacks a no-arg constructor.
- super.method(args) - calls the superclass's implementation of a method. This is commonly used to extend the superclass behaviour: perform some work before or after invoking the superclass version.

Common mistake: Trying to call super.super.method() - Java does not allow jumping to a grandparent's implementation. You can only directly access the immediate superclass.

8.9 final Classes and Methods

A final class prevents any class from extending it. This is used for security (e.g., String to prevent mutable subclasses) and for performance (allows the JIT to apply more aggressive optimisations, like devirtualising all calls). A final method cannot be overridden in subclasses. This allows inlining by the compiler because the method implementation is known at compile time (though the JIT can inline non-final methods too, but final methods are a stronger guarantee).

When to use: Mark utility classes (with only static methods) as final to prevent useless subclassing. Mark methods in template-method patterns as final if their algorithm must not be changed.

8.10 Abstract Classes

An abstract class is a class that is declared with the abstract modifier. It may contain abstract methods (method declarations without a body) and concrete methods. You cannot instantiate an abstract class directly; it must be subclassed, and the subclass must implement all abstract methods.

Abstract classes are useful when you have shared state (fields) and partial implementation that subclasses should reuse. For example, a Figure abstract class might have a color field and a concrete getColor method, but an abstract area method. Compare to interfaces, which cannot have instance fields (though they can have static fields).

8.11 Interfaces

An interface defines a contract. Before Java 8, it could only contain abstract methods and constants. Since Java 8, interfaces can also contain:

- default methods: Provide a default implementation that classes can use or override. This was introduced to allow adding new methods to existing interfaces without breaking all implementors. The classic example is Iterable.forEach.

- static methods: Like class static methods, they belong to the interface and are invoked via InterfaceName.method().
- private methods (Java 9): Allow sharing code between default methods within the interface.

A class implementing an interface must provide implementations of all abstract methods. With multiple interfaces, if two interfaces define default methods with the same signature, the implementing class must resolve the conflict by overriding the method. It can delegate to one of the defaults with InterfaceName.super.method().

Marker interfaces: Interfaces with no methods (like Serializable, Cloneable) are called marker interfaces. They indicate some property that the class promises to satisfy. Annotation-based markers are now more common.

8.12 Multiple Interface Implementation vs Single Class Inheritance

Java's class inheritance is single, but interface implementation is multiple. This design avoids the state-related diamond problem. Two interfaces can have fields, but they are implicitly public static final, so they don't introduce instance state. Default methods can conflict, but the compiler forces explicit resolution. This gives the flexibility of multiple inheritance of type (and behaviour) without the complexity of inheriting implementation from multiple sources.

8.13 Polymorphism

Polymorphism is the ability of a reference of a parent type to refer to an object of a child type and invoke behaviour specific to that child. The JVM achieves dynamic dispatch by looking up the method in the object's actual class's vtable, not the reference's declared type.

Compile-time vs runtime: The compiler checks that the method exists in the reference type (or one of its supertypes). At runtime, the actual implementation of the object's class is called. This is what makes design patterns like Strategy, Observer, and Factory work - the client code works with an interface, and the implementation can be swapped without changing the client.

Inline caching: Modern JVMs use polymorphic inline caches to speed up virtual calls. The first time a call site is executed, the JIT records the concrete type. A guard checks that type on subsequent calls; if it matches, a direct call is made (inlined). If multiple types are seen, a small decision tree is built. This makes virtual calls almost as fast as direct calls in monomorphic (single receiver) call sites.

8.14 Casting

Upcasting (assigning a subclass reference to a superclass variable) is always safe and implicit. Downcasting (telling the compiler to treat a reference as a more specific type) requires an explicit cast and a runtime check. If the object is not an instance of that type, a ClassCastException is thrown.

```
Object obj = "hello";
String s = (String) obj;        // fine
Integer i = (Integer) obj;      // ClassCastException at runtime
```

With pattern matching for instanceof (Java 16+), you can avoid the explicit cast:

```
if (obj instanceof String s && s.length() > 5) {
    System.out.println(s.toUpperCase()); // s is in scope and typed
        String
}
```

The variable s is in scope where it is definitely assigned, eliminating the possibility of a separate casting error.

8.15 instanceof with Pattern Matching (Java 16+)

This feature combines the type check and the variable declaration. The pattern variable is implicitly final. Its scope is limited to the statements where the test is true (including the right side of &&). You can use it in complex conditions; the compiler will ensure the variable is available only when the match succeeds.

In Java 21, this is extended to switch expressions with pattern matching and when guards, enabling very concise data-driven dispatch.

8.16 The Object Class

Every class ultimately inherits from Object. Its methods are the foundation of the Java ecosystem.

equals(Object): Override to provide logical equality. The contract from the Javadoc is strict: it must be reflexive, symmetric, transitive, and consistent. Violating transitivity in inheritance hierarchies is a classic design problem. The standard recipe is to use instanceof to check type, then compare significant fields. Objects.equals(a, b) simplifies field-by-field comparison.

hashCode(): Must return the same integer for equal objects. If you override equals but not hashCode, hash-based collections (like HashSet, HashMap) will malfunction: equal objects can land in different buckets and be treated as distinct. Always override both together. Use Objects.hash(field1, field2) for a convenient implementation (though it boxes primitives; for high performance, compute manually).

toString(): Returns a string representation, used for logging and debugging. The default implementation prints ClassName@hexHashCode, which is rarely useful. Always override to include relevant state.

clone(): A problematic mechanism. The Cloneable interface is a marker, and Object.clone() is a native method that performs a shallow copy. If your class contains mutable reference fields, you must deep-copy them manually. Even Joshua Bloch recommends using copy constructors or static factory methods instead.

finalize(): Deprecated in Java 9 and removed for removal. Finalizers run unpredictably, if at all, and can resurrect objects. Never use them for resource cleanup; use try-with-resources or Cleaner (Java 9+).

8.17 Records (Java 16+)

A record is a restricted form of class that models immutable data. It automatically generates a canonical constructor, accessor methods named after the components (e.g., x() instead of getX()), equals, hashCode, and toString. All fields are private final.

Records cannot extend other classes (they implicitly extend Record), but they can implement interfaces. They cannot have instance fields beyond the components declared in the record header. This guarantees that the automatically generated equals and hashCode cover all instance state.

The compact constructor provides a place to validate invariants without repeating the component list. You can also define additional constructors, but they must delegate to the canonical constructor. Records are the idiomatic way to write data carrier classes and eliminate an enormous amount of boilerplate.

8.18 Sealed Classes (Java 17+)

Sealed classes and interfaces allow you to control which classes can extend or implement them. Using the permits clause, you list the allowed subtypes. Each permitted subclass must be either final, sealed, or non-sealed (which re-opens the hierarchy for extension).

This feature is the foundation for exhaustive pattern matching. When you switch over a sealed type, the compiler can verify that all permitted subclasses are covered, eliminating the need for a default case and turning maintenance oversights (adding a new subtype but forgetting to handle it somewhere) into compile-time errors.

8.19 Enums

enum types are special kinds of classes with a fixed set of constant instances. Each constant is a public static final field of the enum type. The compiler automatically generates values() (returns all constants) and valueOf(String) (converts from string). Enum constructors are implicitly private; you cannot instantiate enums via new.

Enums can have fields, methods, and even implement interfaces. This makes them extremely powerful for things like state machines, strategy patterns, and command dispatchers.

EnumSet and EnumMap: These are highly optimized collection implementations that internally use bit vectors (for sets) or arrays (for maps) indexed by the enum's ordinal. They are orders of magnitude faster than general-purpose HashSet/HashMap for enum keys.

Singleton via enum: Because the JVM ensures only one instance of each enum constant exists (serialization safety included), an enum with a single constant is the simplest way to implement a thread-safe singleton.

Chapter 9

Memory and Object Lifecycle

9.1 The Heap and the Stack

Every Java thread has its own stack, which stores frames. A frame corresponds to a method invocation and holds:

- The local variable array (includes method parameters, this for instance methods).
- An operand stack (where intermediate values are pushed and operations pop).
- A reference to the runtime constant pool of the current method's class.

The stack is private to the thread, so no synchronisation is needed. Allocation on the stack is extremely fast - just moving the stack pointer. The heap is the shared memory area where all objects and arrays reside. All threads can access heap objects if they have a reference.

Primitive variables declared inside methods live directly on the stack. Object references live on the stack, but the objects themselves live on the heap. When a method returns, its frame is popped, and the local references disappear. If those were the only references to an object, the object becomes eligible for garbage collection.

9.2 Object Creation Cost

Creating an object involves several steps:

1. Memory allocation: The JVM allocates a contiguous block of memory on the heap. In HotSpot, this is extremely fast - typically a pointer bump in a Thread-Local Allocation Buffer (TLAB). TLABs are small per-thread buffers allocated from the heap; allocation inside a TLAB is just incrementing a pointer, requiring no synchronisation. 2. Zeroing: All fields are set to their default values (zero, false, null). This is often done at TLAB allocation time. 3. Header initialization: The mark word is set up (identity hash code not yet assigned), and the klass pointer is set. 4. Constructor invocation: The constructor body executes, which may include complex logic, field initialisers, and superclass constructor calls.

While allocation itself is cheap, the cumulative effect of object creation includes increased garbage collection frequency. Short-lived objects (typically those created in a method and never escaping) are cheap because they are collected during a minor GC (which is fast). Long-lived objects that survive many GC cycles are more expensive because they must be copied between survivor spaces and eventually promoted to the old generation, where collection is costlier.

9.3 Garbage Collection

Java's GC is a tracing garbage collector. The process starts from GC roots:

- Local variables and parameters of active stack frames.

- Static fields of loaded classes.
- JNI (native) references.
- Monitor objects (used for synchronization).

The GC marks all objects reachable from these roots by traversing reference chains. Any object not marked is unreachable and can be reclaimed. The actual reclamation strategy depends on the algorithm.

Mark-and-sweep is the conceptual model: mark reachable objects, then sweep (free) the rest. Modern collectors do much more, including copying and compacting to avoid fragmentation.

9.4 Generational GC

Most Java objects die young. The generational hypothesis states that object lifetimes follow a bimodal distribution: many objects become garbage within milliseconds, while a small fraction live for the entire application. The generational collector partitions the heap into the young generation (Eden + two survivor spaces S0, S1) and the old generation.

Minor GC: When Eden fills up, a minor GC is triggered. Live objects are copied from Eden and one survivor space to the other survivor space. After several such copies (determined by the tenuring threshold), surviving objects are promoted to the old generation. Minor GCs are typically fast because they copy only live objects (which are few) and the spaces are small.

Major GC / Full GC: When the old generation is full, a major GC runs. This involves marking the entire heap and compacting the old generation, which is a stop-the-world operation that can take seconds on large heaps. Tuning the old generation size and monitoring promotion rates is essential for avoiding frequent full GCs.

9.5 GC Algorithms

- Serial GC (-XX:+UseSerialGC): single-threaded, stop-the-world. Suitable for small heaps (< 100 MB) and client-side applications.
- Parallel GC (-XX:+UseParallelGC): multi-threaded for both young and old generation collections, still stop-the-world. High throughput, good for batch processing where pauses are tolerable.
- G1 GC (-XX:+UseG1GC, default since Java 9): divides the heap into equal-sized regions. It performs concurrent marking to identify regions with most garbage (hence ‘‘Garbage First’’). Evacuation of these regions is stop-the-world but brief. G1 aims to meet a pause time target (-XX:MaxGCPauseMillis, default 200 ms). It is the best general-purpose collector for heaps from a few hundred MB to many GB.
- ZGC (-XX:+UseZGC): a low-latency collector that does almost all work concurrently, including compaction. It uses colored pointers (64-bit pointers with metadata bits) to track object state. Pauses are in the sub-millisecond range, independent of heap size. Ideal for heaps from a few GB to multi-TB.

- Shenandoah (-XX:+UseShenandoahGC): similar goals to ZGC, using forwarding pointers (Brooks pointers) instead of colored pointers. Also concurrent compaction.

Selecting a GC depends on your latency budget. For most services, G1 is sufficient. For p99 latency under 1 ms, ZGC or Shenandoah is mandatory.

9.6 Soft, Weak, and Phantom References

Java's reference classes allow finer control over object reachability.

- SoftReference: The referent is cleared only when the JVM is running out of memory. Useful for memory-sensitive caches where you want to keep objects as long as memory is plentiful, but let them go under pressure.
- WeakReference: The referent is cleared at the next GC cycle after it becomes only weakly reachable. The canonical use is WeakHashMap: keys are held via weak references. When the key is no longer strongly reachable, the entry is automatically removed from the map after the next GC. This is perfect for canonical maps where you want to associate metadata with objects without preventing their garbage collection.
- PhantomReference: The referent is never directly accessible; get() always returns null. When the referent becomes phantom reachable, the phantom reference is enqueued in a ReferenceQueue. This allows you to schedule cleanup actions *after* the object has been finalized and is about to be reclaimed, but without resurrecting it. Used for replacing finalizers with Cleaner.

Reference queues: All reference types can be associated with a queue upon construction. The GC will enqueue the reference object when the referent's reachability changes, allowing the application to poll and perform cleanup.

9.7 Memory Leaks in Java

Java memory leaks happen when objects that are no longer needed by the application logic are still reachable from GC roots, so the GC cannot reclaim them. Classic causes:

- Static collections: A static HashMap that you put entries into but never remove. The entries stay forever because the map itself is a GC root (via static field).
- Listeners not deregistered: An observer pattern where the subject holds strong references to listeners. If listeners are short-lived but never unregistered, they and everything they reference are pinned.
- Inner classes: Non-static inner classes hold an implicit reference to their outer class instance. If the inner class instance is retained (e.g., in a collection), the outer class is also retained. This is a subtle but common source of leaks, especially with anonymous classes used as listeners.
- Caches: A cache implemented as a HashMap with strong keys and values will grow indefinitely. Use a size-bounded cache or weak/soft references.

- ThreadLocals: Thread-local variables that are not cleaned up can leak memory in thread-pooled environments where threads are reused.

Diagnosing leaks involves taking a heap dump (jmap -dump:live,file=heap.bin <pid> or -XX:+HeapDumpOnOutOfMemoryError) and analyzing it with tools like Eclipse Memory Analyzer (MAT) that show the paths from GC roots to the retained objects.

9.8 try-with-resources and AutoCloseable

The AutoCloseable interface has a single method close() throws Exception. Any resource that must be released (streams, connections, file handles, locks) should implement it. The try-with-resources statement automatically calls close() at the end of the block, equivalent to a finally block.

```
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    return br.readLine();
}
```

The compiler transforms this into:

```
BufferedReader br = new BufferedReader(new FileReader(path));
Throwable primary = null;
try {
    return br.readLine();
} catch (Throwable t) {
    primary = t;
    throw t;
} finally {
    if (br != null) {
        if (primary != null) {
            try { br.close(); } catch (Throwable suppressed) {
                primary.addSuppressed(suppressed);
            }
        } else {
            br.close();
        }
    }
}
```

This ensures that if both the body and close() throw exceptions, the close() exception is added as a *suppressed exception* to the primary exception, keeping the original failure as the main cause while still recording the close failure.

Always use try-with-resources for any AutoCloseable. It eliminates resource leak bugs and makes code cleaner.

Chapter 10

Generics

10.1 Why Generics Exist

Before Java 5 (Java 1.4 and earlier), collections were untyped. A List held Object instances. When you retrieved an element, you had to cast it manually:

```
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0);
```

If someone added an Integer by mistake, you would get a ClassCastException at runtime, possibly far from the point of insertion. Generics were introduced to enforce compile-time type checking. With generics, the compiler ensures that only String objects are added to a List<String>, and no cast is needed when retrieving elements. Bugs are caught at compile time, and the code becomes self-documenting.

10.2 Generic Classes and Methods

A generic class declares one or more type parameters in angle brackets after the class name. These parameters act as placeholders for actual types.

```
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
```

A generic method has its own type parameters, placed before the return type:

```
public static <T> T firstOrNull(List<T> list) {
    return list.isEmpty() ? null : list.get(0);
}
```

Type inference allows you to call firstOrNull(stringList) without specifying <String> explicitly.

Naming convention: Single uppercase letters - E (element), K (key), V (value), T (type), S, U, V for secondary types.

10.3 Bounded Type Parameters

Sometimes you need to restrict a type parameter to a particular type or its subtypes. Use the extends keyword (even for interfaces).

```
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}
```

This bounds T to any type that implements Comparable<T>. Multiple bounds are possible: <T extends Number & Serializable>. The first bound can be a class; subsequent must be interfaces.

10.4 Wildcards

Wildcards add flexibility to generics by allowing unknown types.

- List<?>: a list of an unknown type. You cannot add any element except null because the compiler doesn't know what type is safe. You can read elements as Object.
- List<? extends Number>: a list of some subtype of Number. You can read a Number, but you cannot add (except null) because you don't know the exact subtype - adding an Integer might be unsafe if the list is actually List<Double>. This is a *producer* wildcard.
- List<? super Integer>: a list of some supertype of Integer. You can add Integer safely, but reading gives Object. This is a *consumer* wildcard.

Wildcards make API signatures more flexible without sacrificing type safety.

10.5 PECS Principle

Producer Extends, Consumer Super. If a parameterized type produces values (you read from it), use ? extends T. If it consumes values (you write to it), use ? super T. If both reading and writing are needed, use the exact type T. This maximises the usability of your API.

For example, Collections.copy(dest, src) has signature public static <T> void copy(List<? super T> dest, List<? extends T> src). The source list is a producer of T (extends), the destination list is a consumer (super).

10.6 Type Erasure

Generics are a compile-time only feature. During compilation, the Java compiler performs *type erasure*:

- Replaces all type parameters with their leftmost bound (or Object if unbounded).
- Inserts casts where necessary to maintain type safety.
- Generates bridge methods to preserve polymorphism.

The JVM sees only raw types. List<Integer> and List<String> become List at runtime. This ensures backward compatibility with pre-generic code, but it has consequences:

- You cannot overload methods that differ only by generic parameter types (they have the same erasure).
- You cannot use instanceof with parameterized types (e.g., obj instanceof List<String> does not compile) because the runtime has no generic information.
- You cannot create an instance of a type parameter (new T()) because the runtime doesn't know what T is.

The Class object itself represents the raw type; you can get List.class but not List<String>.class.

10.7 Raw Types

A raw type is a generic type used without specifying type parameters, like List instead of List<String>. They exist solely for backward compatibility with code written before Java 5. The compiler issues an unchecked warning when raw types are used because they bypass generic type checking.

Never use raw types in new code. They can cause heap pollution: a List<String> reference passed to a method taking raw List can have an Integer added to it, and the error will only surface later as a ClassCastException at a far-away location.

10.8 Generic Arrays

Java generics and arrays don't mix well. You cannot create an array of a parameterized type: new T[10] is illegal because the runtime doesn't know T. You also cannot create an array of a parameterized concrete type: new List<String>[10] is illegal (generic array creation error). This is because arrays are covariant (a String[] is an Object[]) while generics are invariant, and the combination breaks type safety.

The workaround is to create an array of the raw type and cast:

```
@SuppressWarnings("unchecked")
T[] array = (T[]) new Object[size];
```

But this is unsafe and should be avoided. A better approach is to use a List<T> instead of an array. If an array is absolutely necessary, keep the class private and ensure the array is never exposed to the outside.

In practice, the JDK itself uses this pattern in ArrayList:

```
transient Object[] elementData; // non-generic array
// Methods cast elements to (E) when reading.
```

Chapter 11

Collections Framework

11.1 The Collection Hierarchy

The Java Collections Framework is built on a set of core interfaces that define the contract for different types of data structures. At the top is Iterable, which allows an object to be the target of the enhanced for-each loop. Collection extends Iterable and provides the bulk operations: add, remove, contains, size, clear, and so on. From Collection, three main subinterfaces branch out:

- List: an ordered collection that permits duplicates and provides positional access via an integer index. Implementations include ArrayList and LinkedList.
- Set: a collection that cannot contain duplicate elements, modeled after the mathematical set abstraction. Implementations include HashSet (no order), LinkedHashSet (insertion order), and TreeSet (sorted order).
- Queue: a collection designed for holding elements prior to processing. Subinterface Deque adds double-ended operations. Implementations include LinkedList, ArrayDeque, and PriorityQueue.

Map is not a Collection because it maps keys to values, but it is still part of the framework. Its key-set, value-collection, and entry-set are views that implement Set and Collection. This interface hierarchy gives you the vocabulary to describe almost any data structure need.

Common mistake: treating all collections as interchangeable when Big-O behavior differs dramatically. A List with O(1) random access allows fast get(i), while a Set does not support index-based retrieval at all. Using the wrong collection can turn an O(1) operation into O(n) or O(log n) without any compiler warning.

11.2 ArrayList

ArrayList is the workhorse of the Collections Framework. It stores elements in a contiguous Object[] array. The array reference is the only field (plus a size counter). When you create new ArrayList<>(), the backing array is a shared zero-length array constant (to save memory) until the first element is added. The first add allocates an array of capacity 10 (the default). Subsequent additions that exceed the capacity trigger a resize: a new array is created with capacity = old capacity + (old capacity » 1), i.e., 1.5x growth, and the old elements are copied via System.arraycop

Performance: Random access via get(i) and set(i, elem) is O(1). Adding at the end via add(e) is amortized O(1) because the copy cost is spread over many insertions. Inserting or removing at an arbitrary index is O(n) because all elements to the right must be shifted. The Iterator's remove() is also O(n) because it shifts elements; using removeIf can be more efficient (it marks and then compacts in one pass).

Memory: Each ArrayList object has 12-16 bytes of header, a reference to the array (4 or 8 bytes), and an int size (4 bytes). The array itself has overhead

(16 bytes) plus 4 or 8 bytes per slot (for references). For large lists, the array may be partially empty, wasting memory. Calling trimToSize() reduces the array to the exact size of the elements, but a subsequent add will cause another allocation.

Production advice: If you know the approximate size upfront, use the constructor new ArrayList<>(expectedSize). This avoids multiple array copies. For read-only lists, prefer List.of which uses a compact internal array and is truly immutable.

```
List<String> list = new ArrayList<>(1000); // pre-allocate
list.add("a");
list.get(0); // O(1)
list.add(0, "b"); // O(n): shifts all elements
```

Common mistake: using ArrayList in a concurrent context without synchronization. ArrayList is not thread-safe. If multiple threads modify it, you risk ArrayIndexOutOfBounds or silent data corruption. Use CopyOnWriteArrayList for read-heavy concurrent access, or wrap with Collections.synchronizedList.

11.3 LinkedList

LinkedList is a doubly-linked list. Each node is a private static inner class storing the element, a prev pointer, and a next pointer. The list maintains references to the head and tail nodes. Random access via get(i) traverses the list from the end nearest to the index, making it O(n/2) on average, i.e., O(n). Insertion at the beginning or end is O(1) because it just updates the head/tail pointers and the adjacent node's pointers.

Why LinkedList is usually the wrong choice: Despite its theoretical O(1) insertion at the head/tail, its poor cache locality and high constant overhead per node make it slower than ArrayList even for workloads that insert at the beginning. Each node is a separate object, scattered on the heap, so iterating a LinkedList causes many cache misses. Modern CPU prefetchers cannot predict the next node's location. Moreover, each node consumes 24 bytes of overhead plus the element reference, so memory usage is significantly higher than an ArrayList of the same size.

When to use: If you need to frequently add and remove elements in the middle of a list during iteration (via ListIterator), LinkedList is the only option that supports O(1) structural changes at the current iterator position. But ArrayDeque is superior for queues and stacks because it has array-based O(1) operations at both ends without the pointer overhead.

11.4 ArrayDeque

ArrayDeque is a resizable circular array that implements Deque. It stores elements in a single Object[] array, with head and tail indices that wrap around modulo the array length. Adding at the head decreases the head index; adding at the tail increases the tail index. When the head and tail meet, the array is doubled and the elements are copied in correct order.

Performance: All deque operations (addFirst, addLast, removeFirst, removeLast, peek) are amortized O(1). Because everything is in a single array, iteration is fast and cache-friendly. ArrayDeque is the recommended implementation for stacks (replacing Stack, which is synchronized and uses Vector) and queues (replacing LinkedList).

```
Deque<String> stack = new ArrayDeque<>();
stack.push("first");  // adds at head
stack.push("second");
String top = stack.pop(); // "second"     LIFO

Deque<String> queue = new ArrayDeque<>();
queue.offer("a");  // adds at tail
queue.offer("b");
String head = queue.poll(); // "a"     FIFO
```

Limitations: ArrayDeque does not support concurrent modification. For concurrent usage, ConcurrentLinkedDeque is available. ArrayDeque also does not allow null elements (because null is used as a sentinel to detect the tail slot in some implementations Actually the head element is always present; null is used to signal that poll returns if queue empty, so elements cannot be null because null is the return value for an empty queue. So adding null is prohibited to disambiguate.)

11.5 HashSet and HashMap

HashMap is the most widely used map implementation. It stores key-value pairs in an array of buckets, where each bucket is either a linked list or a balanced tree. The bucket index is computed as (n - 1) & hash, where n is the table size (always a power of two). The hash is derived from the key's hashCode() by spreading higher bits into lower bits (XOR with hash >> 16) to improve distribution when n is small.

Collision resolution: When multiple keys land in the same bucket, they form a linked list (separate chaining). If a bucket's list grows beyond 8 elements and the table size is at least 64, the list is converted to a balanced red-black tree (treeification). This ensures O(log n) worst-case lookup even under hash collision attacks, rather than degrading to O(n). If the tree shrinks to 6 elements, it reverts to a linked list (untreeification).

Resizing: The default load factor is 0.75. When the number of entries exceeds capacity * load factor, the table size is doubled. All entries must be rehashed into the new table, which is an expensive O(n) operation. To avoid multiple resizings when loading a known number of elements, specify an initial capacity large enough to hold them without resizing (or use HashMap.newHashMap(expectedSize) which computes an appropriate initial capacity).

Null handling: HashMap allows one null key (stored in bucket 0) and multiple null values. HashSet is a wrapper around HashMap where the value is a static dummy Object (called PRESENT). Adding an element to a HashSet calls map.put(element, PRESENT).

Iteration order: No guarantees. The order can change when the map is resized. If you need predictable iteration order, use LinkedHashMap or a TreeMap.

Common mistake: Using mutable objects as keys without careful hashCode and equals implementations that remain stable. If a key's hash code changes after it is placed in the map, it becomes unfindable (lost in the wrong bucket). Always use immutable keys or ensure the mutable fields affecting equality are not changed.

11.6 LinkedHashSet and LinkedHashMap

LinkedHashMap extends HashMap by adding a doubly-linked list through all entries. This linked list defines the iteration order, which is either insertion order (default) or access order (when accessOrder constructor parameter is true). In access-order mode, each get or put moves the entry to the end of the list, making it ideal for implementing Least Recently Used (LRU) caches.

LRU cache example:

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxSize;
    LRUCache(int maxSize) {
        super(maxSize, 0.75f, true); // access order
        this.maxSize = maxSize;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > maxSize;
    }
}
```

When the map grows beyond maxSize, the eldest entry (head of the list) is automatically removed. The linked list adds a small memory overhead (two references per entry) and a slight runtime cost for maintaining the list on each access, but for moderate sizes the performance is excellent.

11.7 TreeSet and TreeMap

TreeMap implements a sorted map using a Red-Black tree, a self-balancing binary search tree. This guarantees O(log n) time for get, put, remove, containsKey. The tree is ordered by the keys' natural order (if they implement Comparable) or by a provided Comparator. TreeSet wraps a TreeMap similarly to HashSet.

NavigableMap and NavigableSet: TreeMap implements NavigableMap, which provides powerful range-view operations: floorKey (greatest key given), ceilingKey (least key given), higherKey, lowerKey, and sub-map views like headMap, tailMap, subMap that reflect changes in the original map.

When to use: If you need sorted iteration or range queries, TreeMap is the right tool. Its memory overhead per entry is higher than HashMap because each node stores left/right/parent pointers and a colour flag, in addition to the key and value.

Common mistake: Using TreeMap when only occasional sorted traversal is needed and the dataset is large. HashMap is O(1) average; TreeMap is O(log n). For millions of entries, the constant factor and pointer-chasing of the tree can be significantly slower. Consider using HashMap and sorting the entries only when needed (e.g., via stream().sorted()).

11.8 PriorityQueue

PriorityQueue is an unbounded priority queue based on a binary heap stored in an array. The head of the queue is the least element according to the natural ordering or a provided Comparator. If multiple elements tie for least, the head is arbitrarily chosen among them.

Internal structure: The heap array's element at index 0 is the root (minimum). Children of node i are at 2*i+1 and 2*i+2. offer(e) appends the element to the end of the array and sifts up until the heap invariant is restored (O(log n)). poll() removes and returns the root, then moves the last element to the root and sifts down (O(log n)). peek() returns the root without modifying (O(1)).

Iteration: The iterator does not traverse elements in sorted order; it walks the array sequentially. To retrieve elements in sorted order, repeatedly poll() until the queue is empty, which will destroy the queue.

Not thread-safe: PriorityQueue is not synchronized. For concurrent priority queues, use PriorityBlockingQueue, which is thread-safe and blocks on take() when empty.

11.9 Comparable vs Comparator

Comparable<T> defines a natural ordering for a class. The class implements compareTo(T o) and returns a negative integer, zero, or positive integer. This ordering should be consistent with equals (though not strictly required, it is strongly recommended to avoid confusing behavior in sorted sets/maps). The natural ordering is used when no Comparator is provided.

Comparator<T> is a separate object that defines an ordering. It can be passed to sorting methods or to ordered collections. Java 8 added many static and default methods to Comparator: comparing(Function) extracts a sort key; thenComparing chains comparators; reversed() reverses; nullsFirst/nullsLast handle nulls safely.

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getDept)
                      .thenComparing(Employee::getSalary).reversed()
                      .thenComparing(Employee::getName))
    .collect(Collectors.toList());
```

Best practice: Favor Comparator factory methods over writing custom compareTo implementations whenever you need multiple sort orders. The natural ordering should represent the "obvious" default (e.g., an employee's ID).

11.10 Collections Utility Class

The java.util.Collections class provides a set of static algorithms that operate on collections. Some of the most useful:

- sort(list) and sort(list, comparator): stable sort using TimSort (a hybrid of merge sort and insertion sort) for objects, and dual-pivot quicksort for primitive arrays.
- binarySearch(list, key): O(log n) search on a sorted list.
- min(coll) and max(coll): find extreme elements.
- shuffle(list): randomize order using Fisher-Yates algorithm.
- reverse(list): reverse order in place.
- frequency(coll, obj): count occurrences.

- unmodifiableList(list): returns a read-only view. Any mutating method throws UnsupportedOperationException. The view reflects changes to the backing list, so it is not a snapshot.
- synchronizedList(list): wraps each method call in synchronized. Iteration must be manually synchronized.
- checkedList(list, type): adds runtime type checking to a generic list to prevent heap pollution.

11.11 List.of, Set.of, Map.of (Java 9+)

These static factory methods on List, Set, and Map create truly immutable collections. Unlike Collections.unmodifiableList, which is a view over a mutable list, the collection returned by List.of is an independent, fixed-size, null-hostile container. Its class is an internal type that throws UnsupportedOperationException on any mutation attempt. The implementations are optimized for small sizes: for up to two elements, they use field-based classes; for more, an array-based class is used.

Rules: The collections cannot contain null elements; attempting to pass null will throw NullPointerException. They are value-based, meaning their identity is not guaranteed and should not be relied upon.

```
List<String> immutable = List.of("a", "b", "c");
Set<Integer> numbers = Set.of(1, 2, 3);
Map<String, Integer> map = Map.of("key1", 1, "key2", 2);
```

These are ideal for API boundaries where you want to hand out a collection that can be safely shared without fear of modification.

11.12 Arrays.asList Trap

Arrays.asList returns a fixed-size list backed by the original array. Changes to the list write through to the array, and vice versa. However, the list does not support structural modifications (add, remove) because the backing array cannot be resized. Attempting to call these methods throws UnsupportedOperationException.

```
String[] arr = {"a", "b"};
List<String> list = Arrays.asList(arr);
list.set(0, "c"); // OK, array becomes ["c","b"]
list.add("d");    // throws UnsupportedOperationException
```

Correct usage: If you need a modifiable list from an array, wrap it: new ArrayList<>(Ar
This creates a new list that is independent. If you need an immutable list, prefer List.of(arr) or Arrays.asList(arr) wrapped in Collections.unmodifiableList (but note that the array can still be mutated externally).

Chapter 12

Lambdas and the Stream API

12.1 Anonymous Classes Before Lambdas

Before Java 8, the only way to pass behavior was to define a class that implements an interface and instantiate it, often anonymously. For example, sorting a list by length required:

```
Collections.sort(words, new Comparator<String>() {
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
});
```

The syntax is verbose and obscures the actual logic. Each anonymous class generated a separate .class file at compile time (e.g., Main$1.class), increasing JAR size and class-loading overhead.

Lambdas replace this with a concise expression that the compiler compiles into a invokedynamic instruction, deferring the actual implementation to runtime. No separate .class file is emitted; instead, the JVM uses LambdaMetafactory to generate a functional interface implementation at first use. This makes lambdas both syntactically light and efficient.

12.2 Functional Interfaces

A functional interface is an interface that has exactly one abstract method. The @FunctionalInterface annotation is a compile-time check that the interface satisfies this requirement; it also signals intent to readers. The single abstract method determines the target type for lambda expressions and method references.

Examples: Runnable (void run()), Callable<V> (V call()), Comparator<T> (int compare(T,T)). Even interfaces with multiple default or static methods are functional if they have exactly one abstract method.

The compiler uses the target type to infer the lambda's parameter types. This is why you can write (a, b) -> a - b only when the compiler knows the target type is a Comparator<Integer> (or similar).

12.3 Built-in Functional Interfaces

The package java.util.function provides a comprehensive set of interfaces to cover the most common shapes of lambdas:

- Predicate<T>: boolean test(T t). Used for filtering. Default methods and, or, negate allow combining predicates.
- Function<T,R>: R apply(T t). Transforms a value. Default methods andThen, compose.
- Consumer<T>: void accept(T t). Performs an action. Default method andThen.

- Supplier<T>: T get(). Provides a value, often for lazy instantiation.
- UnaryOperator<T> extends Function<T,T>: unary operation on same type.
- BinaryOperator<T> extends BiFunction<T,T,T>: binary operation on same type.
- BiFunction<T,U,R>: R apply(T t, U u). Two-argument transformation.
- BiConsumer<T,U>, BiPredicate<T,U>.

Primitive specializations: IntPredicate, LongFunction<R>, ToIntFunction<T>, DoubleConsumer, etc. These avoid boxing overhead for the primitive parameter or return value.

12.4 Lambda Syntax

A lambda expression consists of a parameter list, an arrow ->, and a body. If the body is a single expression, its value is returned. If the body is a block, you must use return explicitly. Parameter types can be omitted if they can be inferred.

```
// expression body
Function<String, Integer> lengthFn = s -> s.length();
// block body
Comparator<String> byLength = (a, b) -> {
    int diff = a.length() - b.length();
    return diff;
};
// no parameters
Runnable r = () -> System.out.println("Run");
```

12.5 Method References

Method references are syntactic sugar for lambdas that already exist as named methods. There are four kinds:

1. Static method: ClassName::staticMethod e.g., Integer::parseInt.
2. Instance method of a specific object: instance::method e.g., System.out::println.
3. Instance method of an arbitrary object of a particular type: ClassName::instanceMethod e.g., String::length - the first parameter becomes the receiver.
4. Constructor: ClassName::new e.g., ArrayList::new.

Method references are often more readable than the equivalent lambda, and they avoid the indirection of a lambda wrapper.

12.6 Closure Semantics

Lambdas can access variables from the enclosing scope, but with a crucial restriction: the variables must be *effectively final* - i.e., their value is never reassigned after initialization. The lambda captures the value of the variable, not a reference to the variable. For objects, the captured value is the reference to the object; the object itself can be mutated.

```
int threshold = 5; // effectively final
List<String> filtered = words.stream()
    .filter(s -> s.length() > threshold)
    .collect(Collectors.toList());
```

This restriction exists because lambdas are not implemented as inner classes that directly access the stack; the variable's value is copied into the synthetic method. If the variable were mutable, the lambda might see a stale value or the mutation would not be visible to the enclosing scope. The "effectively final" rule avoids this confusion.

12.7 Stream Creation

Streams can be obtained from many sources:

- From a collection: collection.stream() (sequential) or collection.parallelStream().
- From an array: Arrays.stream(array).
- From a specific set of values: Stream.of("a", "b", "c").
- From a range: IntStream.range(0, 100).
- Infinite streams: Stream.iterate(seed, UnaryOperator) (infinite), Stream.iterate(seed, predicate, next) (finite in Java 9+), Stream.generate(Supplier).
- From a file: Files.lines(path).
- From a BufferedReader: bufferedReader.lines().

A stream is a lazy sequence of elements that can be consumed only once. After a terminal operation is invoked, the stream is closed and cannot be reused.

12.8 Intermediate Operations

Intermediate operations are lazy: they return a new stream and are not executed until a terminal operation is called. They build a pipeline of operations.

- filter(Predicate): keeps only elements matching the predicate.
- map(Function): transforms each element.
- flatMap(Function): transforms each element into a stream and flattens the results into a single stream. Useful for one-to-many transformations.
- distinct(): eliminates duplicates (uses equals and hashCode).
- sorted() and sorted(Comparator): sorts elements.
- peek(Consumer): allows a side effect (e.g., logging). Use only for debugging; never rely on it for production logic because it is non-deterministic in parallel streams.
- limit(n): truncates the stream to at most n elements.

- skip(n): discards the first n elements.
- takeWhile(Predicate) (Java 9): takes elements while predicate is true, then stops.
- dropWhile(Predicate) (Java 9): drops elements while predicate is true, then passes through the rest.

Streams are lazily evaluated, so operations like filter and map are fused into a single pass over the data when possible.

12.9 Terminal Operations

Terminal operations trigger the pipeline and produce a result or side effect. After a terminal operation, the stream is consumed.

- forEach(Consumer): applies an action to each element. Order is not guaranteed for parallel streams; use forEachOrdered if needed.
- collect(Collector): reduces elements into a mutable container (List, Set, Map, etc.).
- reduce(identity, accumulator): reduces elements to a single value (e.g., sum).
- count(): returns the number of elements.
- min(Comparator) / max(Comparator): returns the minimum/maximum element wrapped in Optional.
- findFirst() / findAny(): returns the first or any element (for parallel, findAny is more efficient).
- anyMatch, allMatch, noneMatch: short-circuiting boolean checks.

12.10 Collectors in Depth

The Collectors utility class provides many predefined collectors.

toList(): returns a mutable ArrayList. No guarantees on type, mutability, or thread-safety. For a specific list type, use toCollection(ArrayList::new).

toSet(): returns a mutable HashSet. For sorted sets, use toCollection(TreeSet::new).

toMap(keyMapper, valueMapper): collects into a HashMap. If duplicate keys occur, an IllegalStateException is thrown. Use the overload with a merge function to resolve duplicates. For a specific map type, there is a four-argument version.

groupingBy(classifier): groups elements into a Map<K, List<V>>. You can provide a downstream collector for the values (e.g., groupingBy(Employee::getDept, counting())). Supports multi-level grouping via another groupingBy downstream.

partitioningBy(predicate): splits into two groups (Map<Boolean, List<V>>).

joining(delimiter, prefix, suffix): concatenates CharSequence elements.

collectingAndThen(collector, finisher): applies a final transformation to the collector's result.

12.11 Optional<T>

Optional is a container that may or may not contain a non-null value. It is designed to be used as a method return type, clearly signaling that the value might be absent and forcing the caller to handle that case.

Creating: Optional.of(value) (throws NPE if value is null), Optional.ofNullable(value) (safe), Optional.empty(). Transforming: map(Function), flatMap(Function) (if the mapping returns an Optional). Filtering: filter(Predicate). Consuming: ifPresent(Cor ifPresentOrElse(Consumer, Runnable). Providing defaults: orElse(default) (eager evaluation, always creates the default), orElseGet(Supplier) (lazy, only evaluated if empty), orElseThrow(Supplier).

Avoid misuse: Optional should not be used as a field, method parameter, or collection element. It is not Serializable and adds overhead. Use it only for return values.

12.12 Parallel Streams

A parallel stream splits the source into multiple substreams, processes them concurrently using the common ForkJoinPool, and then combines the results. You obtain a parallel stream with parallel() or parallelStream().

When it helps: Large datasets (millions of elements) with independent, CPU-intensive operations that do not rely on encounter order.

When it hurts: Small streams (splitting and combining overhead dominates). Operations that block (e.g., I/O) because they tie up the common pool threads. Operations that rely on mutable shared state or encounter order (implicit sorting or findFirst forces ordering). The common pool has a default parallelism equal to the number of cores minus one; blocking operations can starve other parallel tasks.

Customizing parallelism: You can submit a parallel stream task to a custom ForkJoinPool to control thread count and avoid blocking the common pool.

Chapter 13

Exception Handling

13.1 The Throwable Hierarchy

The root class for all errors and exceptions is Throwable. It has two direct subclasses:

- Error: serious problems that a reasonable application should not try to catch. Includes OutOfMemoryError, StackOverflowError, NoClassDefFoundError. These usually indicate JVM failures or resource exhaustion; recovery is impossible.
- Exception: application-level exceptions. It splits into RuntimeException (unchecked) and all other Exception subclasses (checked).

Checked exceptions must be declared in the method signature with throws or caught within the method. Unchecked exceptions (RuntimeException and its subclasses, plus Error) do not require declaration.

13.2 Checked vs Unchecked

The original Java philosophy was that checked exceptions would force developers to handle error conditions, leading to more robust software. In practice, the result was often:

```
try {
    // ...
} catch (IOException e) {
    e.printStackTrace(); // or empty catch
}
```

This swallows the exception and provides no recovery. Checked exceptions also break composability in lambdas and streams, because most functional interfaces do not throw checked exceptions.

Modern Java libraries (Spring, Hibernate) overwhelmingly use unchecked exceptions, wrapping checked exceptions in runtime equivalents. The consensus: use checked exceptions only when the caller can realistically recover (e.g., a file not found prompting the user for a different filename). Otherwise, use unchecked exceptions.

13.3 try/catch/finally

A try block must be followed by at least one catch or a finally block. If a finally block is present, it is always executed, even if the try block returns or throws. If both the try block and finally block throw exceptions, the finally exception replaces the original unless the original is saved as a suppressed exception.

Common mistake: placing a return inside finally. This overrides any return value from the try block and swallows exceptions that occurred in try or catch. Avoid return, break, or continue inside finally.

13.4 Multi-catch

Since Java 7, you can catch multiple exception types in a single catch clause: catch (IOException | SQLException e). The exception parameter is implicitly final. This eliminates duplicate code when the handling logic is identical. The exception hierarchy is still respected: if a caught supertype is already listed, the subtype cannot be listed.

13.5 try-with-resources

The try-with-resources statement automatically closes resources that implement AutoCloseable. Resources are declared in parentheses and are closed in reverse order of declaration. If the body throws and closing also throws, the closing exception is added to the primary exception via addSuppressed. This preserves the original exception while still recording the close failure.

```
try (BufferedReader br = new BufferedReader(new
    FileReader("file.txt"))) {
     String line = br.readLine();
     // ...
}
```

The compiler generates a complex nested try-catch-finally that handles normal and exceptional closure. Always use try-with-resources for streams, connections, locks; it eliminates resource leaks.

13.6 throw and throws

throw instantiates and throws a Throwable. The stack trace is captured at the point of throw (by calling fillInStackTrace()). throws is part of the method signature and declares that the method may propagate checked exceptions. It is a compile-time check, not a runtime enforcement. A method can declare checked exceptions it does not actually throw (often for future compatibility or superclass compatibility).

13.7 Exception Chaining

When you catch a low-level exception and throw a higher-level one, pass the original as the cause: throw new ServiceException("Operation failed", e). The full stack trace of the cause is preserved and can be retrieved with getCause(). Most logging frameworks print the entire causal chain. Without chaining, the root cause is lost.

13.8 Custom Exceptions

Define custom exception classes when you need to convey specific error information that the standard exceptions don't capture. Extend RuntimeException for unchecked, or Exception for checked. Add fields like error codes, request IDs, or payloads. Always provide constructors that accept message, cause, and both.

13.9 Common Anti-patterns

- Catching Exception or Throwable: masks fatal errors like OutOfMemoryError. Use specific exception types.
- Empty catch block: silently swallows the exception, making debugging impossible. If the exception is truly expected, at least log a debug message.
- Log and rethrow: catching an exception, logging it, and rethrowing it causes duplicated stack traces and confusion. Either handle it (and don't rethrow) or let it propagate.
- Exceptions for control flow: throwing and catching exceptions is expensive (stack trace capture). Use them only for exceptional conditions, not for regular program logic.

Chapter 14

Input and Output

14.1 java.io Overview

The java.io package is organized around the decorator pattern. At the core are the abstract classes InputStream and OutputStream for byte streams, and Reader and Writer for character streams. Concrete classes like FileInputStream provide the basic implementation, and decorators like BufferedInputStream add functionality (buffering).

The API is verbose but flexible. It has been largely superseded by java.nio.file for file operations and the HttpClient for network I/O, but it remains fundamental for understanding Java's I/O model.

14.2 FileInputStream, FileOutputStream, etc.

FileInputStream reads bytes from a file. Without buffering, each read() call triggers a native OS I/O operation, which is extremely slow. Always wrap with BufferedInputStream for performance. For reading text, FileReader uses the platform's default charset, which is dangerous for portable applications. Instead, use new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF$_8$).BufferedReader*provides*readLine(),*aconvenientmetho*

Writing: FileWriter and BufferedWriter, or better, Files.writeString(path, content, StandardCharsets.UTF$_8$).

14.3 PrintWriter and System.out

System.out is a PrintStream (byte-oriented). PrintWriter is character-oriented and offers the same formatting methods. Neither throws IOException on write errors; instead, they set an internal error flag. Call checkError() to detect failure. For production code, use a proper logging framework (SLF4J, Logback) rather than printing directly.

14.4 java.nio.file

The java.nio.file package, introduced in Java 7, provides the modern file API centered on the Path interface and the Files utility class. Path is obtained via Paths.get("dir", "file.txt") or Path.of("dir/file.txt"). It abstracts the file system path in a platform-independent manner.

Key Files methods:

- readString(Path, Charset): reads entire file into a String.
- writeString(Path, CharSequence, Charset, OpenOption...): writes a string to a file, creating or truncating.
- readAllLines(Path, Charset): returns a List<String>.
- lines(Path, Charset): returns a Stream<String> for lazy line processing.

- walk(Path, int depth, FileVisitOption...): returns a Stream<Path> of the directory tree, lazily evaluated.
- createDirectories(Path): creates all nonexistent parent directories.
- copy(Path, Path, CopyOption...): copies files or directories.
- move(Path, Path, CopyOption...): moves or renames.

Files methods throw IOException; they are checked and force handling. Using try-with-resources is not needed for these static methods (they open and close internally), but for streaming methods like lines() and walk(), you must close the returned Stream (preferably via try-with-resources) to release the underlying file handle.

14.5 Serialization

Java's built-in serialization is a mechanism to convert an object graph into a byte stream (ObjectOutputStream) and back (ObjectInputStream). A class must implement Serializable (marker interface) to be serializable. The serialVersionUID field controls version compatibility. Fields marked transient are skipped.

Security and maintainability issues: Java serialization has a long history of vulnerabilities. Deserializing untrusted data can lead to arbitrary code execution. It is also fragile: changes to class structure often break compatibility. The industry has largely moved to data-interchange formats like JSON, XML, or Protocol Buffers. Do not use Java serialization for new systems unless you have a specific, controlled requirement.

14.6 JSON with Jackson

Jackson is the most widely used JSON library in the Java ecosystem. Its core is ObjectMapper, which provides:

```
ObjectMapper mapper = new ObjectMapper();
// Serialization
String json = mapper.writeValueAsString(object);
// Deserialization
MyClass obj = mapper.readValue(json, MyClass.class);
```

ObjectMapper is thread-safe after configuration, so it should be reused as a singleton. It uses annotations like @JsonProperty to customize field names, @JsonIgnore to exclude fields, and @JsonCreator for custom constructors. Modules like jackson-datatype- add support for java.time types. For large data, Jackson provides a streaming API (JsonParser, JsonGenerator) that avoids materializing the entire tree in memory.

14.7 JSON with Gson

Gson, from Google, offers a simpler API: Gson gson = new Gson(); String json = gson.toJson(obj); MyClass obj = gson.fromJson(json, MyClass.class);. It handles generics with TypeToken. Gson has a smaller footprint and is easier for straightforward serialization. However, Jackson's performance, streaming, and tree model make it more suitable for complex or high-throughput scenarios. Spring Boot defaults to Jackson, so it's the default choice in that ecosystem.

Chapter 15

Concurrency

15.1 Thread Creation

A thread can be created by extending Thread and overriding run(), or by passing a Runnable to the Thread constructor. The recommended way is to implement Runnable (or use a lambda) because it separates the task from the thread execution, allowing better composability and testing.

```
Thread t = new Thread(() -> {
    System.out.println("Running in " +
       Thread.currentThread().getName());
});
t.start(); // do NOT call run() directly     that runs synchronously
```

Calling run() directly executes the method in the current thread; start() creates a new OS thread and then calls run() in that thread.

15.2 Thread Lifecycle

Threads transition through states: NEW (before start()), RUNNABLE (ready or executing), BLOCKED (trying to enter a synchronized block/method), WAITING (wait(), join() without timeout), TIMED_WAITING (sleep(), wait(timeout), join(timeout)), and TERMINATED. JVM tools like jstack show these states for each thread. Understanding them is crucial for diagnosing hung threads.

15.3 Race Conditions

A race condition occurs when the correctness of a program depends on the relative timing of thread execution. The classic example is incrementing a shared counter with count++. This is not atomic; it's three separate operations: load current value, increment, store new value. If two threads interleave these steps, one increment can be lost.

```
int counter = 0;
// Thread A: load 0, increment to 1, store 1 (before B stores)
// Thread B: load 0 (before A stores), increment to 1, store 1
// Result: counter = 1 instead of 2
```

Race conditions are notoriously hard to reproduce and debug. The only correct approach is to use proper synchronization.

15.4 synchronized Keyword

synchronized provides mutual exclusion. It can be applied to methods or blocks. Every object has an intrinsic monitor lock. When a thread enters a synchronized block, it acquires the lock on the given object; other threads trying to acquire the same lock block until it is released.

Reentrancy: A thread can acquire the same lock multiple times (e.g., a synchronized method calling another synchronized method on the same object). The lock is released when the outermost synchronized block exits.

Visibility: Exiting a synchronized block flushes cached variables to main memory; entering a synchronized block invalidates the thread's cache, forcing it to reread from main memory. This establishes a happens-before relationship.

Performance: Contended locks cause thread suspension and rescheduling, which is expensive. Uncontended locks are cheap (biased locking or thin locks in HotSpot). Java 15 deprecated biased locking; Java 21 removed it.

15.5 volatile Keyword

volatile guarantees visibility: a write to a volatile field immediately becomes visible to all other threads, and a read sees the most recent write. It also prevents instruction reordering around the volatile access (memory barrier). However, volatile does not provide atomicity for compound actions like count++. Use it for flags, reference assignments, and other simple state where atomicity is not required.

```
volatile boolean stopped = false;
// Thread 1: while (!stopped) { ... }
// Thread 2: stopped = true; // immediately visible to Thread 1
```

15.6 Deadlock

Deadlock occurs when two or more threads each hold a lock and wait for the others to release their locks. A classic scenario:

```
Object lock1 = new Object();
Object lock2 = new Object();
// Thread A
synchronized(lock1) { synchronized(lock2) { ... } }
// Thread B
synchronized(lock2) { synchronized(lock1) { ... } }
```

Thread A holds lock1 and waits for lock2; Thread B holds lock2 and waits for lock1. Neither can proceed. Prevention: always acquire locks in a consistent global order. Detection: thread dumps show threads in BLOCKED state, with "waiting to lock" and "locked" information. The JVM can detect deadlocks on intrinsic locks and will report them in thread dumps.

15.7 java.util.concurrent ExecutorService

ExecutorService abstracts thread management. Instead of manually creating threads, you submit tasks to an executor, which maintains a pool of worker threads and a task queue.

- Executors.newFixedThreadPool(n): fixed number of threads; excess tasks wait in an unbounded queue.
- Executors.newCachedThreadPool(): creates threads as needed, reusing idle threads; suitable for many short-lived tasks.

- Executors.newSingleThreadExecutor(): single worker thread executing tasks sequentially; guarantees ordering.
- Executors.newScheduledThreadPool(n): supports delayed and periodic task execution.

Always shut down the executor with shutdown() (initiates orderly shutdown) and awaitTermination() (waits for completion). shutdownNow() attempts to stop actively executing tasks.

15.8 Future and Callable

Callable<V> is like Runnable but returns a result and can throw checked exceptions. Submitting a Callable to an ExecutorService returns a Future<V>. future.get() blocks until the task completes and returns the result. To avoid indefinite blocking, use get(timeout, unit). cancel(boolean mayInterruptIfRunning) attempts cancellation; isDone() and isCancelled() report status.

15.9 ThreadPoolExecutor Internals

ThreadPoolExecutor is the concrete implementation behind the Executors factory methods. It has:

- corePoolSize: the target number of threads to keep alive, even if idle.
- maximumPoolSize: the maximum number of threads allowed.
- workQueue: the queue used to hold tasks before they are executed. The choice of queue determines the executor's behavior: SynchronousQueue (direct handoff, used by cached pool), LinkedBlockingQueue (unbounded, fixed pool), ArrayBlockingQueue (bounded).
- RejectedExecutionHandler: called when the queue is full and max threads reached. Policies: AbortPolicy (throws exception), CallerRunsPolicy (runs task in caller's thread), DiscardPolicy (silently discard), DiscardOldestPolicy.

Understanding these parameters is vital for avoiding resource exhaustion or task rejection under load. For production, prefer a bounded queue with a well-defined rejection policy.

15.10 Atomic Classes

The java.util.concurrent.atomic package provides lock-free, thread-safe operations on single variables using hardware-level compare-and-swap (CAS) instructions. AtomicInteger AtomicLong, AtomicBoolean, AtomicReference offer methods like get, set, incrementAndGet(), compareAndSet(expect, update).

CAS works by atomically comparing the current value with an expected value; if they match, it updates to the new value and returns true. If not, it returns false and the caller can retry in a loop. This avoids the weight of intrinsic locks.

For high-contention accumulators, LongAdder and DoubleAdder provide better throughput by distributing the value across multiple internal cells, reducing contention.

15.11 ConcurrentHashMap

ConcurrentHashMap allows concurrent reads and segmented concurrent writes. In Java 7, it used a segment-level lock array. Since Java 8, it uses a node-based design: the table is an array of Node objects; most updates use CAS on the bucket head, with synchronized blocks only for tree bins or when a bucket is being resized. This provides extremely high concurrency.

Atomic compound operations: methods like putIfAbsent, remove(key, value), replace(key, old, new), computeIfAbsent, computeIfPresent, merge. These eliminate the check-then-act race conditions that plague regular HashMap.

size() is an estimate (it sums the sizes of each bin without locking). For exact size, use mappingCount() (returns long).

15.12 CountDownLatch, CyclicBarrier, Semaphore, Phaser

- CountDownLatch: initialized with a count. Threads call await() to block until the count reaches zero. Other threads call countDown() to decrement. A one-shot latch; cannot be reset.
- CyclicBarrier: a group of threads waits at a barrier; when the last thread arrives, all are released, and an optional barrier action runs. Reusable after all threads are released.
- Semaphore: maintains a set of permits. acquire() takes a permit, blocking if none available; release() returns a permit. Useful for controlling access to limited resources (connection pools).
- Phaser: a more flexible barrier that supports dynamic party count and multiple phases.

15.13 BlockingQueue

BlockingQueue is the core of the producer-consumer pattern. ArrayBlockingQueue is bounded and uses a single lock for both put and take. LinkedBlockingQueue is optionally bounded and uses two locks (head and tail) for higher throughput. put(e) blocks when full; take() blocks when empty; offer(e, timeout, unit) and poll(timeout, unit) provide timed alternatives.

15.14 CompletableFuture

CompletableFuture<T> implements both Future<T> and CompletionStage<T>. It represents a future result that can be explicitly completed. More importantly, you can chain asynchronous computations without blocking:

```
CompletableFuture.supplyAsync(() -> fetchUser(userId))
    .thenApply(user -> user.getName())
    .thenAccept(name -> System.out.println("Name: " + name))
    .exceptionally(ex -> { logger.error("Failed", ex); return null;
       });
```

Key methods: thenApply (synchronous transformation), thenCompose (async flat-map), thenCombine (combine two futures), allOf (wait for all), anyOf (wait for any).

By default, async stages execute in the common ForkJoinPool, but you can supply a custom Executor. join() is like get() but throws unchecked exceptions.

15.15 Virtual Threads (Java 21)

Virtual threads are lightweight threads managed by the Java runtime, not the OS. They are instances of Thread that run on a small pool of platform (OS) threads. When a virtual thread performs a blocking I/O operation, it is unmounted from its platform thread, which can then execute another virtual thread. This allows millions of concurrent virtual threads with minimal memory overhead (a few hundred bytes per virtual thread vs. 1 MB for a platform thread's stack).

Virtual threads make blocking I/O cheap again, eliminating the need for reactive programming in many cases. You can write simple, synchronous-style code that scales to high concurrency. They are created via Thread.ofVirtual().start(task) or an executor Executors.newVirtualThreadPerTaskExecutor().

Structured concurrency (preview) groups related virtual threads into a scoped unit, ensuring that all subtasks complete or are cancelled together, similar to structured programming but for concurrency.

Chapter 16

The Module System (Java 9+)

16.1 Why Modules

Before Java 9, the classpath was a flat list of JARs and directories. There was no way to enforce that a package was internal to a library; any class on the classpath could access any public class in any package. This led to widespread use of internal APIs like com.sun.* and sun.misc.*, which hindered the evolution of the JDK. Modules provide strong encapsulation: a module explicitly exports some packages and keeps others hidden.

Modules also enable reliable configuration: dependencies between modules are declared, and the JVM can detect missing modules at startup rather than failing with ClassNotFoundException at runtime. Additionally, jlink can build minimal runtime images containing only the required modules, reducing deployment size.

16.2 module-info.java

A module is defined by a module-info.java file at the root of the source tree:

```
module com.example.myapp {
    requires java.sql;                // uses java.sql module
    requires transitive com.example.util; // re-exports to consumers
    exports com.example.myapp.api;    // public API
    opens com.example.myapp.model to com.fasterxml.jackson.databind;
       // reflection access
    provides com.example.spi.Plugin with com.example.myapp.MyPlugin;
    uses com.example.spi.Plugin;
}
```

The requires statements define module dependencies. exports makes a package accessible to other modules. opens allows deep reflection (including private members) for frameworks. provides ... with declares a service implementation; uses declares a service dependency (for the ServiceLoader).

16.3 Named, Automatic, Unnamed Modules

- Named module: has a module-info.java. Full encapsulation and dependency information.
- Automatic module: a JAR without module-info placed on the module path. Its name is derived from the JAR filename (or Automatic-Module-Name manifest entry). It exports all packages and can read all other modules. It serves as a bridge for non-modularized dependencies.
- Unnamed module: all classes on the classpath are lumped into a single unnamed module. It can read all named modules but cannot be required by named modules. This allows gradual migration: your modular JAR can depend on classpath JARs (which end up in the unnamed module), but not vice versa.

16.4 Classpath vs Module Path

The module path (-module-path) is the new classpath for modules. The JVM resolves modules from the module path and enforces access rules. The classpath still works for backward compatibility, but all classes there go into the unnamed module. Mixing classpath and module path is possible but requires care. Named modules cannot declare requires on the unnamed module. To access code on the classpath from a named module, you need to use reflection with -add-opens or create an automatic module JAR.

16.5 –add-opens and –add-exports

These command-line flags weaken module encapsulation at runtime:

- -add-exports <source-module>/<package>=<target-module>: makes the package accessible at compile and runtime, but does not open for deep reflection.
- -add-opens <source-module>/<package>=<target-module>: opens the package for deep reflection.

They are escape hatches for frameworks that need to access internal JDK APIs or application classes. Use them as temporary measures while migrating to proper module descriptors.

16.6 Strong Encapsulation

Starting in Java 16, -illegal-access=deny became the default. This means that reflective access to internal APIs is denied unless the package is explicitly opened. Tools and libraries that relied on sun.misc.Unsafe had to migrate to supported alternatives (e.g., VarHandle and MethodHandles). This was a deliberate, painful but necessary step to secure the platform and allow internal JDK changes without breaking the ecosystem.

Chapter 17

Modern Java Features

17.1 Pattern Matching for instanceof

Java 16 introduced pattern matching for instanceof. The pattern variable is implicitly final and scoped to the true branch. The compiler automatically casts the object to the target type. This eliminates boilerplate and reduces error-prone manual casts.

```
if (obj instanceof String s && s.length() > 5) {
    System.out.println(s.toUpperCase()); // s is String here
}
// s not in scope here
```

17.2 Switch Expressions with Patterns

Java 21 extends switch to support pattern matching on objects. The switch expression can test the runtime type of its selector and extract components. Combined with sealed classes, the compiler enforces exhaustiveness: all permitted subtypes must be covered, or a default clause.

```
String formatted = switch (shape) {
    case Circle c -> "Circle with radius " + c.radius();
    case Rectangle r when r.width() == r.height() -> "Square";
    case Rectangle r -> "Rectangle";
    // no default needed if Shape is sealed covering all cases
};
```

The arrow syntax (case ... ->) prevents fall-through. A block can yield a value with yield. Guarded patterns (when) refine matching.

17.3 Sealed Classes and Exhaustive Switches

Sealed classes and interfaces define a closed set of permitted subtypes using the permits clause. This enables the compiler to check that pattern-matching switches are exhaustive. It replaces the Visitor pattern, providing a type-safe alternative to an open hierarchy where you'd need a default to handle unknown future subtypes.

17.4 Records Deep Dive

Records are nominal tuples. Beyond the basic auto-generation, you can:

- Compact constructor: public Point if (x < 0) throw ...; validates inputs; the compiler assigns parameters to fields after the compact constructor.
- Additional constructors: must delegate to the canonical constructor via this(...).
- Implement interfaces: records can implement any interface, e.g., Comparable.

- Static fields and methods: allowed as in any class.
- Overriding accessors: you can override the automatically generated accessor to return a modified value, but this breaks the invariant that equals and hashCode are based on the record components.

Records are serializable (if all components are serializable) and the serialization uses the canonical constructor, avoiding the security issues of the default serialization mechanism.

17.5 Text Blocks

Text blocks are a lexer feature; they are strings at runtime. The incidental white space algorithm determines the minimum indentation across all lines and strips that amount from each line. The opening """ can be on its own line, or the text can start on the same line, affecting the indentation baseline. Trailing spaces are stripped unless escaped with \s. Line terminators within the block are normalized to \n. Use \ at end of line to suppress the newline.

17.6 String Templates (Java 21 Preview)

String templates introduce a new syntax for embedding expressions in strings: STR."Hello {name}, you are {age} years old". STR is the default processor that interpolates values by calling toString(). FMT is another processor that interprets format specifiers after %. Processors are statically imported; they are actually implementations of StringTemplate.Processor. This is safer than manual concatenation (no injection by default) and more readable than String.format.

17.7 Sequenced Collections (Java 21)

SequencedCollection is a new interface that unifies encounter order across List, SortedSet, LinkedHashSet, and Deque. It provides getFirst(), getLast(), addFirst(E), addLast(E), removeFirst(), removeLast(), and reversed() view. SequencedMap does the same for maps. This eliminates the inconsistency where you had to call list.get(0) but deque.getFirst() to get the first element.

Chapter 18

Standard Library

18.1 java.lang

The java.lang package is automatically imported. Math provides math functions with strict overflow checks via exact methods. StrictMath offers bit-identical results across platforms (slower). System has currentTimeMillis() (wall clock, can jump), nanoTime() (monotonic, for elapsed time), and arraycopy() (native, fast). Runtime gives memory and processor info. Thread and ThreadLocal are part of this package.

18.2 java.time Date/Time API

The java.time package (JSR 310) is a complete replacement for java.util.Date and Calendar. Immutable and thread-safe classes:

- LocalDate, LocalTime, LocalDateTime: without timezone.
- ZonedDateTime: with timezone (ZoneId).
- Instant: an instantaneous point on the time-line (nanoseconds since epoch).
- Duration (time-based, nanosecond precision) and Period (date-based).
- DateTimeFormatter: for parsing and formatting, thread-safe.

All classes use static factory methods (of, now, parse). Arithmetic operations return new instances because they are immutable. Always use java.time; never mix with old date classes unless forced by legacy APIs.

18.3 java.math BigDecimal

BigDecimal provides arbitrary-precision signed decimal numbers. Use BigDecimal.valueOf(0.1) (creates from string "0.1") instead of new BigDecimal(0.1) (which captures the binary fraction). Division requires a rounding mode (e.g., RoundingMode.HALF$_UP$)*orascaletoavo*

18.4 java.net.http HttpClient (Java 11+)

The new HttpClient supports HTTP/1.1 and HTTP/2, synchronous and asynchronous requests, and WebSocket. Create with HttpClient.newBuilder().build(); (reusable, thread-safe). Build HttpRequest with URI, headers, timeout, and method. Send with send(request, BodyHandlers.ofString()) (sync) or sendAsync(request, handler) (async returning CompletableFuture). It replaces HttpURLConnection, which is a painful API.

18.5 java.util.regex

Always compile Pattern once and reuse. Use Matcher.group(name) for named groups. Flags: Pattern.CASE$_I$*NSENSITIVE*,MULTILINE,DOTALL,UNIX$_L$*INES.Forliteralreplacement*,Matcher.quot

Chapter 19

Testing and Build

19.1 JUnit 5

JUnit 5 is the modern testing framework. @Test marks a test; @BeforeEach/@AfterEach run around each test; @BeforeAll/@AfterAll (static) run once per test class. @DisplayName gives a human-readable name. @Disabled skips a test. @Tag categorizes tests for selective execution.

19.2 JUnit 5 Assertions

assertEquals (with optional delta for floats), assertNotEquals, assertTrue, assertNull, assertSame. assertThrows(Class, Executable) captures the exception. assertAll groups multiple assertions. assertTimeout fails after duration. Use assertThat with Hamcrest or AssertJ for fluent assertions.

19.3 Parameterized Tests

@ParameterizedTest with @ValueSource, @CsvSource, @CsvFileSource, @EnumSource, @MethodSource (returns Stream<Arguments>). The test method parameters match the source. This reduces duplicate test code.

19.4 Mockito

Mockito creates mock objects for unit testing. mock(Class.class) or @Mock (with MockitoExtension). when(mock.method()).thenReturn(value) stubs behavior. verify(mock).meth checks interactions. ArgumentCaptor captures arguments for assertions. @InjectMocks injects mocks into a concrete class. Use mocks to isolate the unit under test; mock external dependencies like databases and network calls.

19.5 Test-Driven Development

TDD cycle: Red (write a failing test) → Green (write minimal code to pass) → Refactor (improve design while keeping tests green). TDD ensures that code is testable and meets requirements. It's a design practice, not just testing.

19.6 Maven Test Lifecycle

During the test phase, the Surefire plugin runs unit tests (class names matching *Test.java, *Tests.java, *TestCase.java). Integration tests run in the verify phase via the Failsafe plugin (names *IT.java, *ITCase.java). Skipping tests: -DskipTests compiles but skips execution; -Dmaven.test.skip=true skips compilation and execution.

19.7 Code Coverage with JaCoCo

JaCoCo instruments bytecode (on-the-fly or offline) and records which lines and branches are executed. Line coverage measures whether a line was hit; branch coverage checks if both branches of conditionals were taken. Aim for high branch coverage; line coverage can be misleading. Exclude generated code and trivial getters/setters. Coverage is not a substitute for good test design.

Chapter 20

Performance and Best Practices

20.1 JVM Startup and Warm-up

JVM startup involves class loading, verification, and interpretation. Peak performance is reached after JIT compilation warm-up, which can take minutes for complex applications. Use -XX:+PrintCompilation to see what is being compiled. For serverless, use GraalVM native image to eliminate warm-up. For long-running servers, provide enough time before load tests. AOT caching (CDS - Class Data Sharing) speeds up startup by sharing class metadata.

20.2 JVM Flags

Important flags:

- -Xmx<size>: max heap (e.g., -Xmx2g).
- -Xms<size>: initial heap; set equal to -Xmx for stable performance.
- -XX:+UseG1GC, -XX:MaxGCPauseMillis=<n>: G1 GC tuning.
- -XX:+PrintGCDetails, -Xlog:gc*: GC logging.
- -XX:MaxMetaspaceSize: limits class metadata area.

Always set heap sizes explicitly; never rely on defaults in production.

20.3 JMH (Java Microbenchmark Harness)

Microbenchmarks are hard: JIT optimizations, dead code elimination, and GC make naive timing incorrect. JMH provides @Benchmark, @Warmup, @Measurement, @Fork to isolate and measure code reliably. Use JMH's guidelines: avoid loops inside benchmarks, return values to prevent dead code elimination, and use Blackhole for side-effect-free code.

20.4 Profiling

Java Flight Recorder (JFR) provides low-overhead event recording. JDK Mission Control visualizes JFR data. Async profiler (open source) gives CPU and allocation flame graphs without safepoint bias. VisualVM is a basic tool for monitoring and heap dumps. Always profile before optimizing; measure, don't guess.

20.5 String Concatenation Performance

Java 9+ compiles string concatenation to invokedynamic with StringConcatFactory, which generates efficient code. Simple concatenations (a + b + c) are optimised. In loops, manual StringBuilder is still better. For building large strings, pre-size the builder.

20.6 Boxing Overhead

Prefer primitive streams (IntStream, LongStream, DoubleStream) over Stream<Integer> etc. Use primitive collections (e.g., Eclipse Collections, fastutil) for large datasets to avoid boxing overhead. Autoboxing can easily cause a 2-4x memory and CPU overhead.

20.7 Effective Java Principles

Joshua Bloch's book is the definitive guide. Key points: prefer composition over inheritance; favor immutability (classes should be final and fields final); use static factory methods instead of constructors (they have names, can cache, can return subtypes); avoid unnecessary object creation (e.g., Boolean.valueOf vs new Boolean); always override hashCode when overriding equals; use try-with-resources; document thread safety. These principles are the bedrock of professional Java code.

20.8 Common Design Patterns

- Builder: solves telescoping constructors; e.g., new Person.Builder().name("John").age(
- Factory Method: subclass decides concrete class; often replaced by Supplier<T> lambdas.
- Strategy: encapsulate algorithm; Comparator is a strategy, easily provided as lambda.
- Observer: publish-subscribe; modern alternatives are reactive streams or ListenableFuture.

Patterns are vocabulary, not mandates. Use them only when they clarify, not complicate.

20.9 Naming Conventions

Consistent naming is a force multiplier for readability. Variables and methods: camelCase (firstName). Classes and interfaces: PascalCase (AccountRepository). Constants: UPPER_SNAKE_CASE (MAX_RETRIES). Package: reverse domain, lowercase (com.example.util). Generic type parameters: single uppercase letter (E, T, K, V). Boolean methods: prefix is, has, can (isEmpty()). Avoid abbreviations except well-known ones. Good naming reduces the need for comments.